

簡介：採用 Flutter 的 Flame

來源：<https://codelabs.developers.google.com/codelabs/flutter-flame-brick-breaker?hl=zh-tw>

步驟數：11

在本程式碼研究室中，您將瞭解如何使用以 Flutter 為基礎建構的遊戲引擎 Flame。您將瞭解 Flame 的元件和效果，以及如何將 Flame 與 Flutter 的狀態管理整合。

目錄

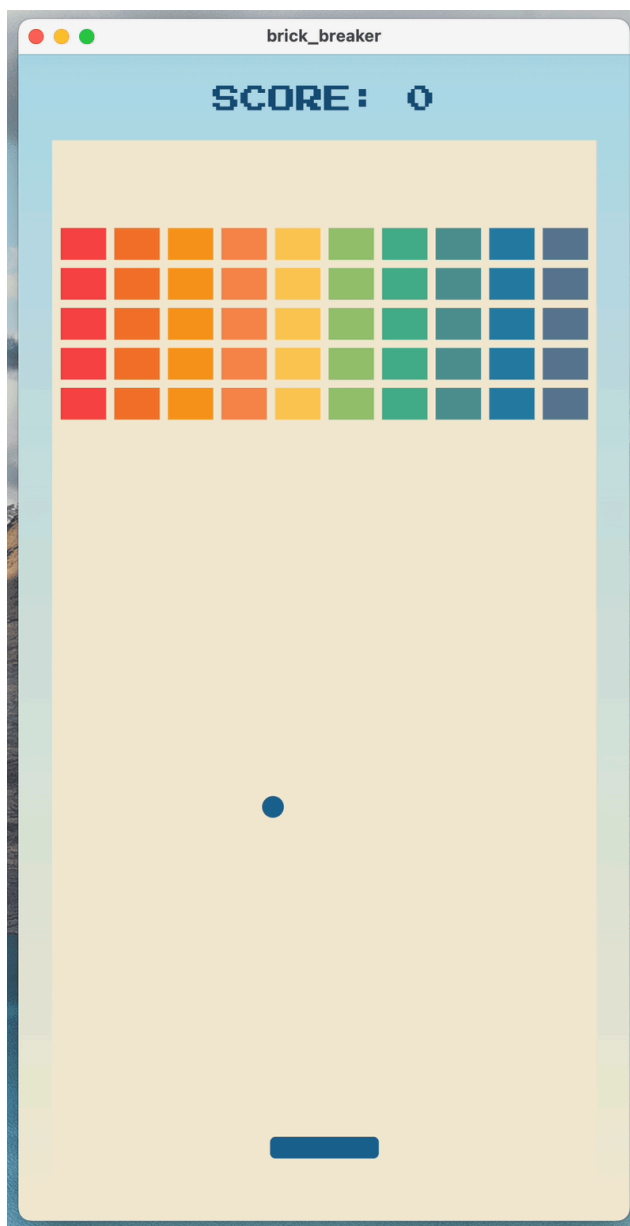
1. [1. 簡介](#)
2. [2. 設定 Flutter 環境](#)
3. [3. 建立專案](#)
4. [4. 建立遊戲](#)
5. [5. 顯示球](#)
6. [6. Bounce around](#)
7. [7. 擊中球](#)
8. [8. 拆除牆壁](#)
9. [9. 贏得遊戲](#)
10. [10. 計算分數](#)
11. [11. 恭喜](#)

步驟 1 · 約 0 分鐘

1. 簡介

Flame 是以 Flutter 為基礎的 2D 遊戲引擎。在本程式碼研究室中，您將建構一款遊戲，靈感來自 1970 年代經典電玩遊戲之一，即 Steve Wozniak 的 [Breakout](#)。您將使用 Flame 的元件繪製球棒、球和磚塊。您將使用 Flame 的 Effects 為蝙蝠的動作製作動畫，並瞭解如何將 Flame 與 Flutter 的狀態管理系統整合。

完成後，遊戲應會像這個動畫 GIF 所示，但速度會稍微慢一點。



課程內容

- 瞭解 Flame 的基本運作方式，首先請參閱 `GameWidget`。
- 如何使用遊戲迴圈。
- 瞭解 Flame 的 `Component` 運作方式。這與 Flutter 的 `Widget` 類似。
- 如何處理衝突。
- 如何使用 `Effect` 為 `Component` 設定動畫。
- 如何在 Flame 遊戲上疊加 Flutter `Widget`。
- 如何將 Flame 與 Flutter 的狀態管理功能整合。

建構項目

在本程式碼研究室中，您將使用 Flutter 和 Flame 建構 2D 遊戲。完成後，遊戲應符合下列規定：

- 在 Flutter 支援的所有六個平台 (Android、iOS、Linux、macOS、Windows 和網頁) 上運作
- 使用 Flame 的遊戲迴圈，維持至少 60 fps。
- 使用 `google_fonts` 套件和 `flutter_animate` 等 Flutter 功能，重現 80 年代電玩遊戲的感覺。

注意：如果您是第一次使用 Flutter，請先完成「[您的第一個 Flutter 應用程式](#)」程式碼研究室。該程式碼研究室會設定 Flutter 開發環境，並協助您開始使用 Flutter。

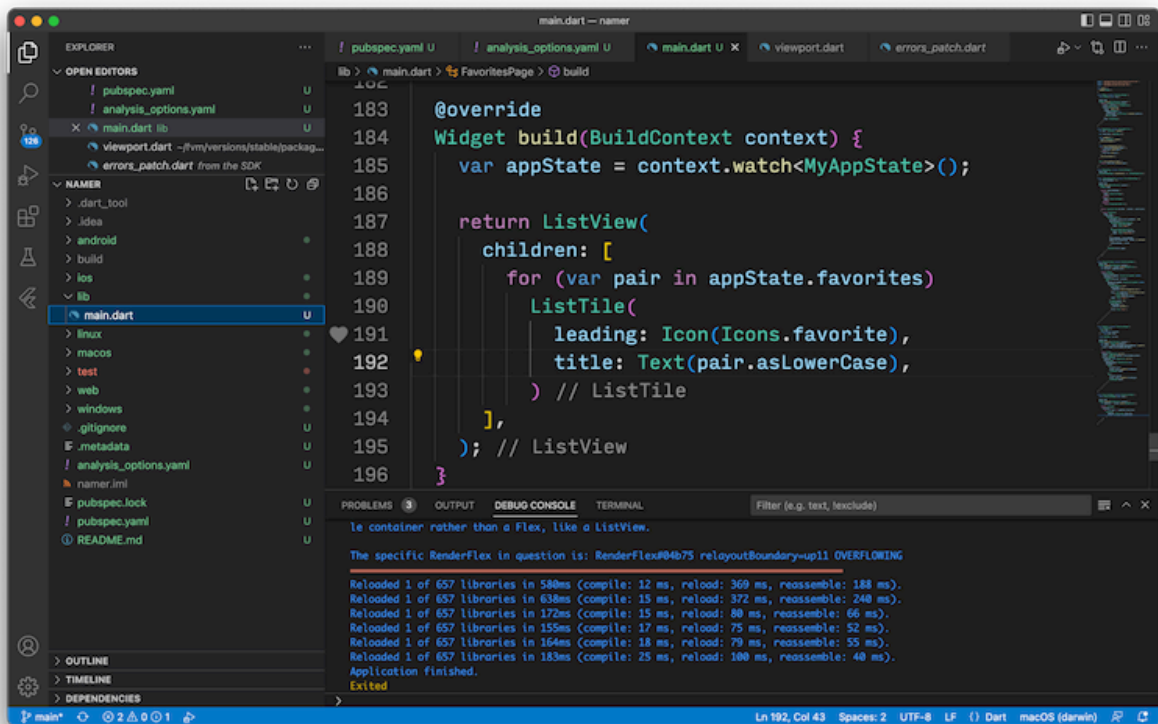
步驟 2 · 約 10 分鐘

2. 設定 Flutter 環境

編輯者

為簡化本程式碼研究室，我們假設您使用 **Visual Studio Code (VS Code)** 做為開發環境。VS Code 免費提供，適用於所有主要平台。本程式碼研究室使用 VS Code，因為操作說明預設為 VS Code 專屬快速鍵。工作變得更簡單：「按一下這個按鈕」或「按下這個鍵來執行 X」，而不是「在編輯器中執行適當動作來執行 X」。

您可以選用任何編輯器，例如 Android Studio、其他 IntelliJ IDE、Emacs、Vim 或 Notepad++，這些編輯器都支援 Flutter。

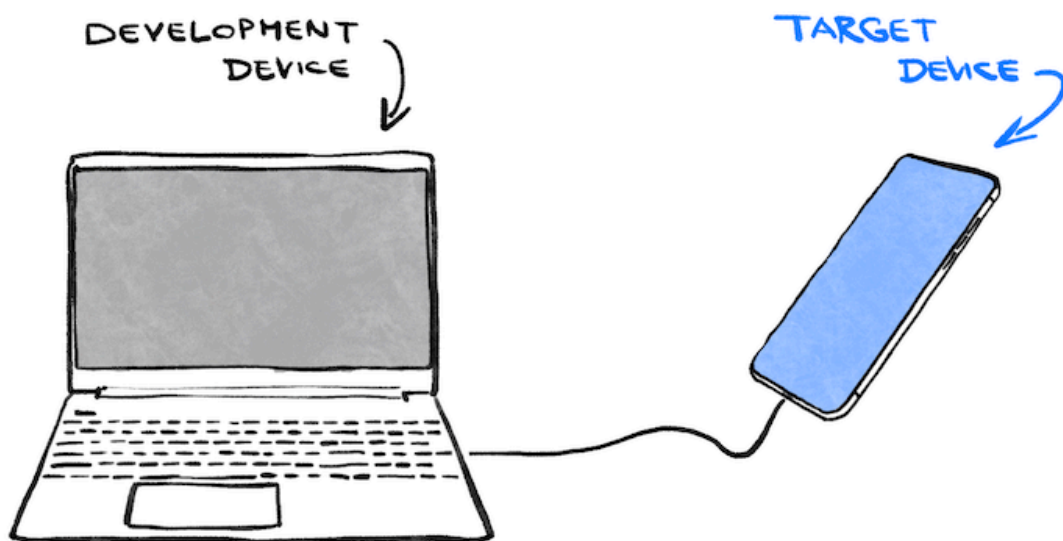


選擇開發目標

Flutter 可為多個平台製作應用程式。您的應用程式可以在下列任一作業系統上執行：

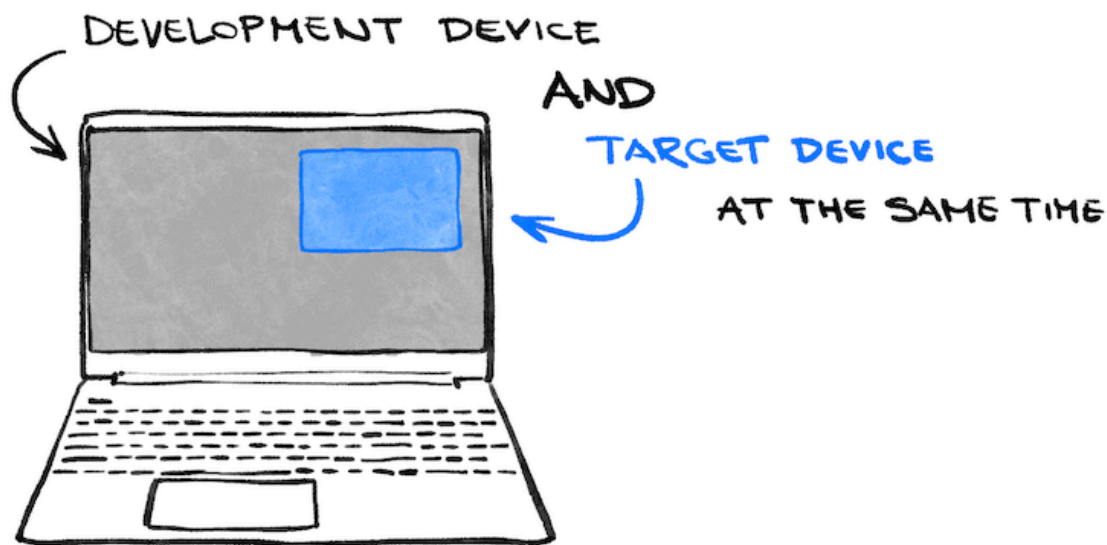
- iOS
- Android
- Windows
- macOS
- Linux
- 網路

一般來說，您會選擇一個作業系統做為開發目標。這個作業系統就是開發過程中用來執行應用程式的 OS。



舉例來說，假設您使用 Windows 筆電開發 Flutter 應用程式，然後選擇 Android 做為開發目標。如要預覽應用程式，請使用 USB 傳輸線將 Android 裝置連接至 Windows 筆電，然後在該 Android 裝置或 Android 模擬器上執行開發中的應用程式。您可能已選擇 Windows 做為開發目標，這會將開發中的應用程式與編輯器一起做為 Windows 應用程式執行。

提示：選擇開發裝置的作業系統做為開發目標。舉例來說，如果您的電腦執行 Windows，請選擇 Windows 做為開發目標。



請先選擇再繼續。您隨時可以在其他作業系統上執行應用程式。選擇開發目標可讓下一個步驟更順利。

安裝 Flutter

如需安裝 Flutter SDK 的最新操作說明，請前往 docs.flutter.dev。

Flutter 網站上的操作說明涵蓋 SDK 安裝作業，以及開發目標相關工具和編輯器外掛程式。請安裝下列軟體，完成本程式碼研究室：

1. Flutter SDK
2. 安裝 Flutter 外掛程式的 Visual Studio Code
3. 所選開發目標的編譯器軟體。(如要以 Windows 為目標，您需要 *Visual Studio*；如要以 macOS 或 iOS 為目標，則需要 *Xcode*)

在下一節中，您將建立第一個 Flutter 專案。

如需排解任何問題，您可能會發現以下 StackOverflow 的問答有助於疑難排解。

常見問題

- [如何找出 Flutter SDK 路徑？](#)
 - [如果找不到 Flutter 指令，該怎麼辦？](#)
 - [如何修正「Waiting for another flutter command to release the startup lock」問題？](#)
 - [如何告知 Flutter Android SDK 的安裝位置？](#)
 - [執行 `flutter doctor --android-licenses` 時發生 Java 錯誤，該如何處理？](#)
 - [如何解決找不到 Android `sdkmanager` 工具的問題？](#)
 - [如何處理「缺少 `cmdline-tools` 元件」錯誤？](#)
 - [如何在 Apple 晶片 \(M1\) 上執行 CocoaPods？](#)
 - [如何在 VS Code 中停用儲存時自動格式化功能？](#)
-

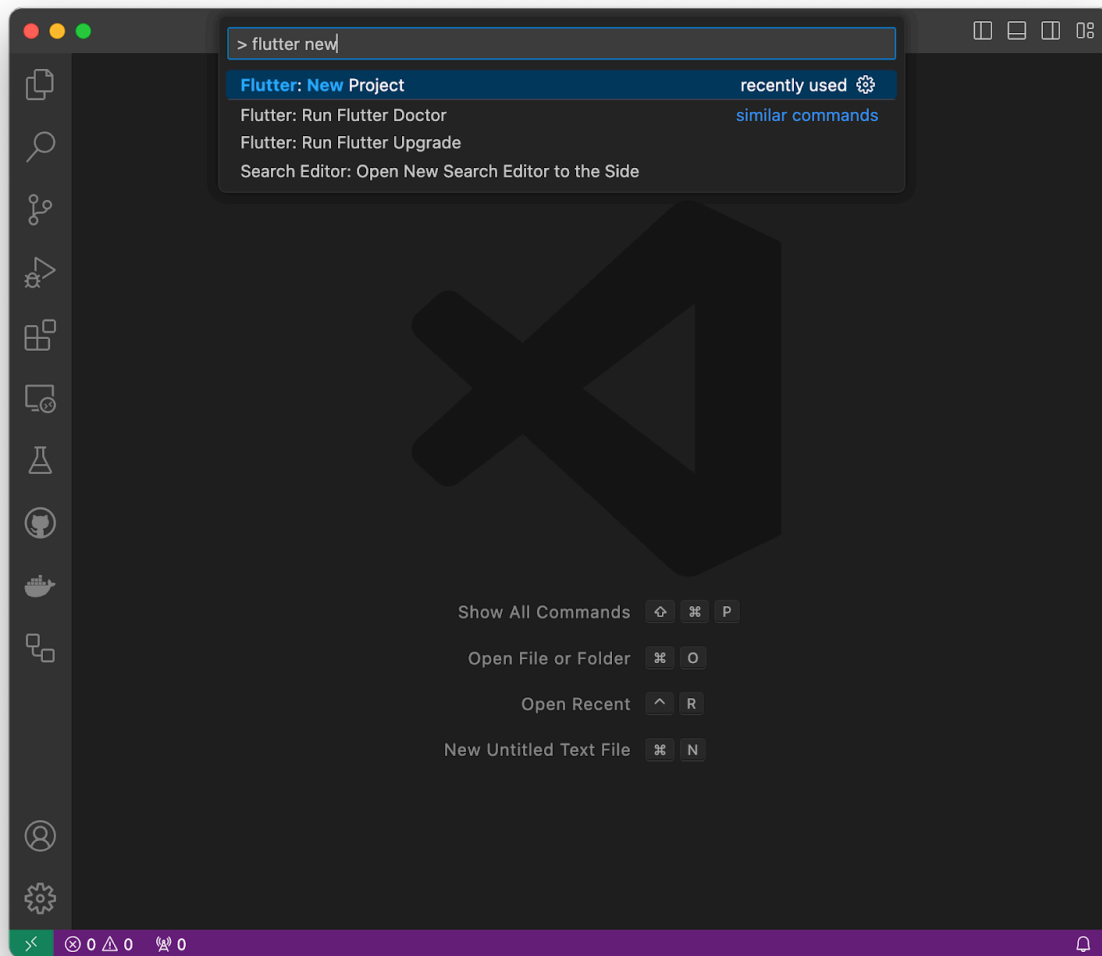
步驟 3 · 約 5 分鐘

3. 建立專案

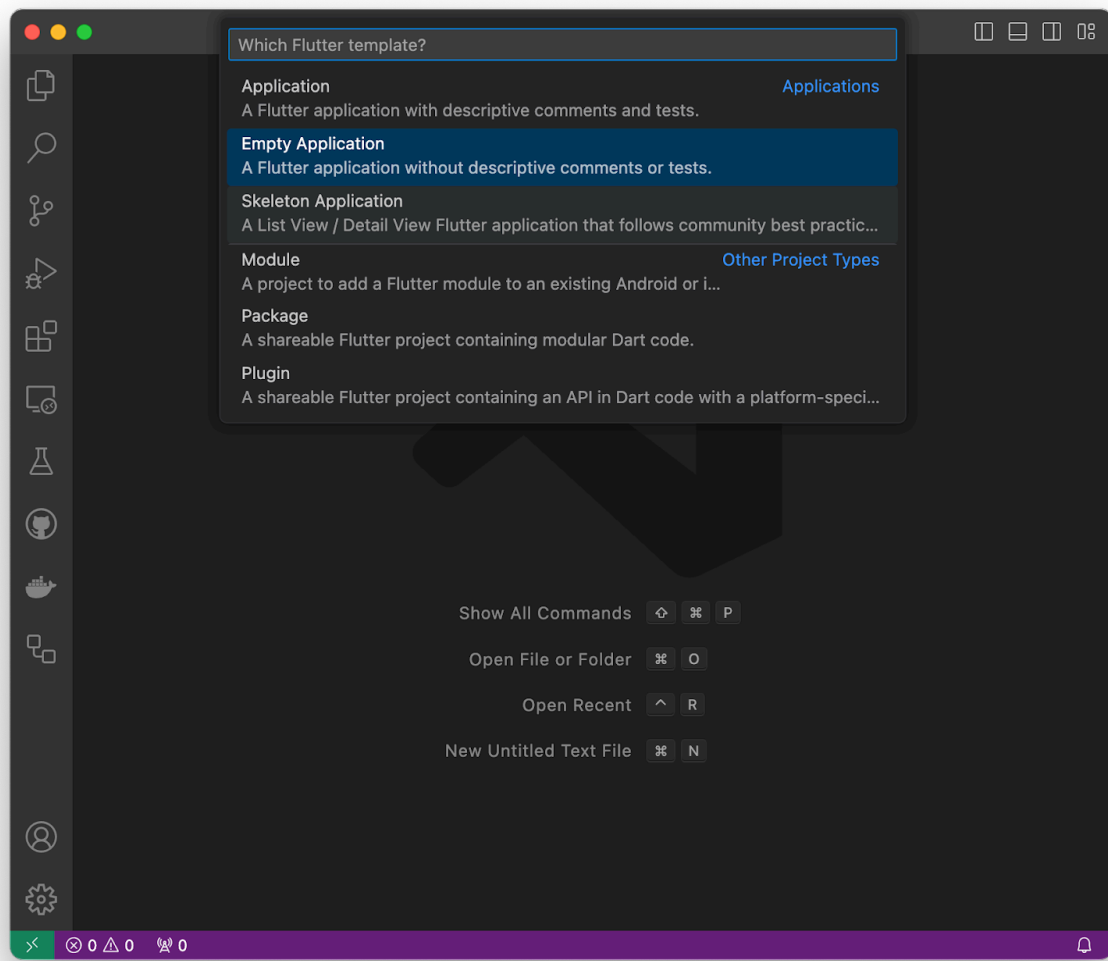
建立第一個 Flutter 專案

包括開啟 VS Code，並在您選擇的目錄中建立 Flutter 應用程式範本。

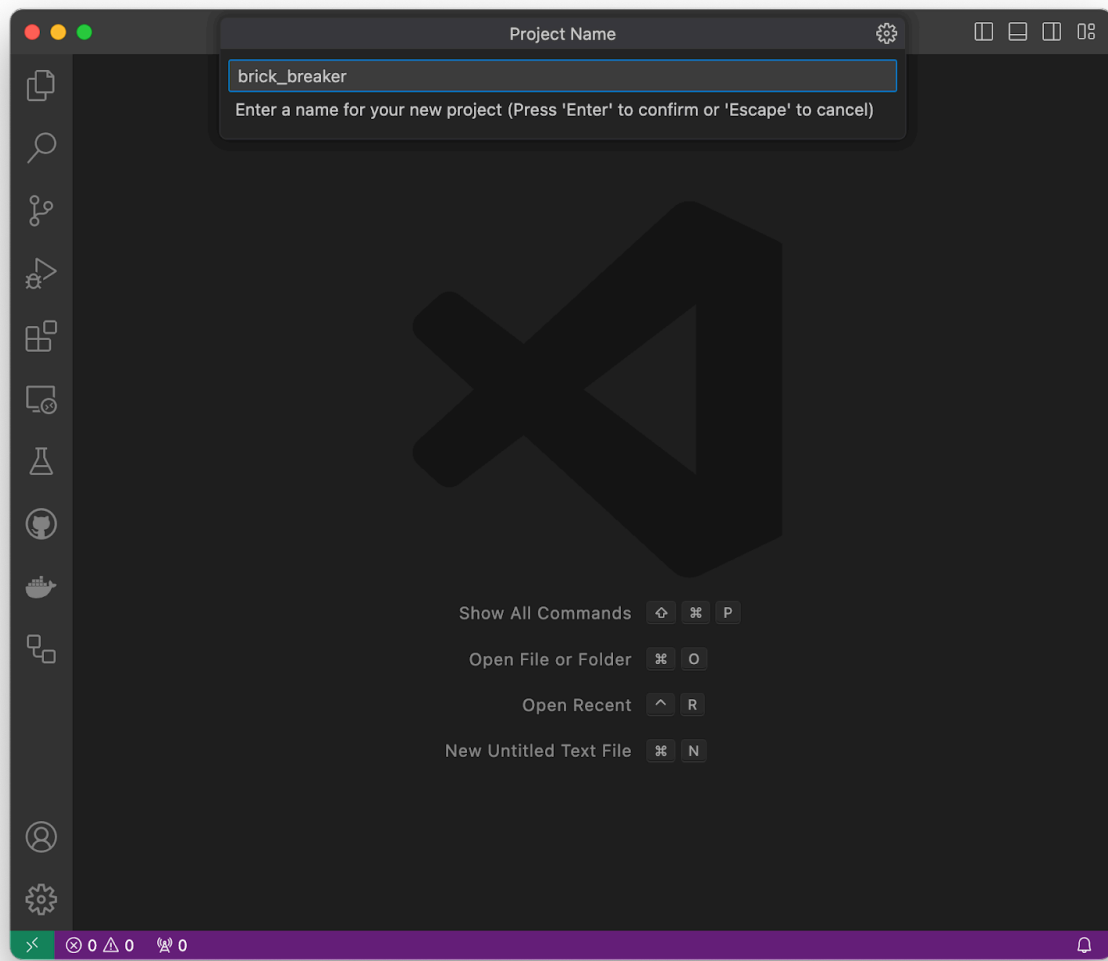
1. 啟動 Visual Studio Code。
2. 開啟指令區塊面板 (**F1** 或 **Ctrl+Shift+P** 或 **Shift+Cmd+P**)，然後輸入「flutter new」。選取「Flutter: New Project」指令。



3. 選取「Empty Application」。選擇要建立專案的目錄。這個目錄不得需要提升權限，路徑中也不得有空格。例如主目錄或 `C:\src\`。



4. 為專案命名 `brick_breaker` 。本程式碼研究室的其餘部分會假設您將應用程式命名為 `brick_breaker` 。

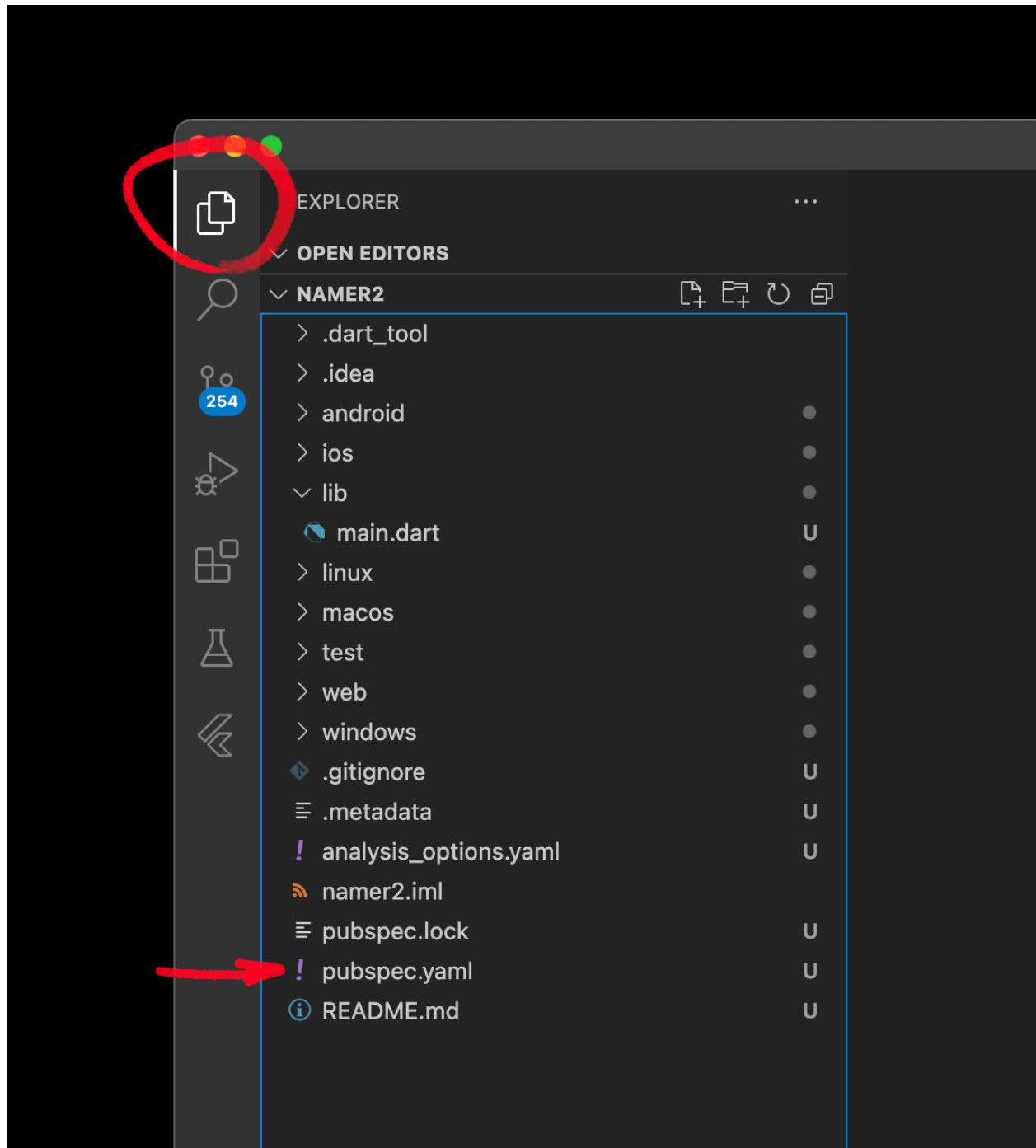


Flutter 現在會建立專案資料夾，並在 VS Code 中開啟該資料夾。現在要以應用程式的基本架構覆寫兩個檔案的內容。

複製並貼上初始應用程式

這會將本程式碼研究室提供的範例程式碼新增至應用程式。

1. 在 VS Code 的左側窗格中，按一下「Explorer」並開啟 `pubspec.yaml` 檔案。



2. 將這個檔案的內容替換成以下內容：

pubspec.yaml

```
name: brick_breaker
description: "Re-implementing Woz's Breakout"
publish_to: "none"
version: 0.1.0

environment:
  sdk: ^3.8.0

dependencies:
  flutter:
    sdk: flutter
  flame: ^1.28.1
  flutter_animate: ^4.5.2
  google_fonts: ^6.2.1

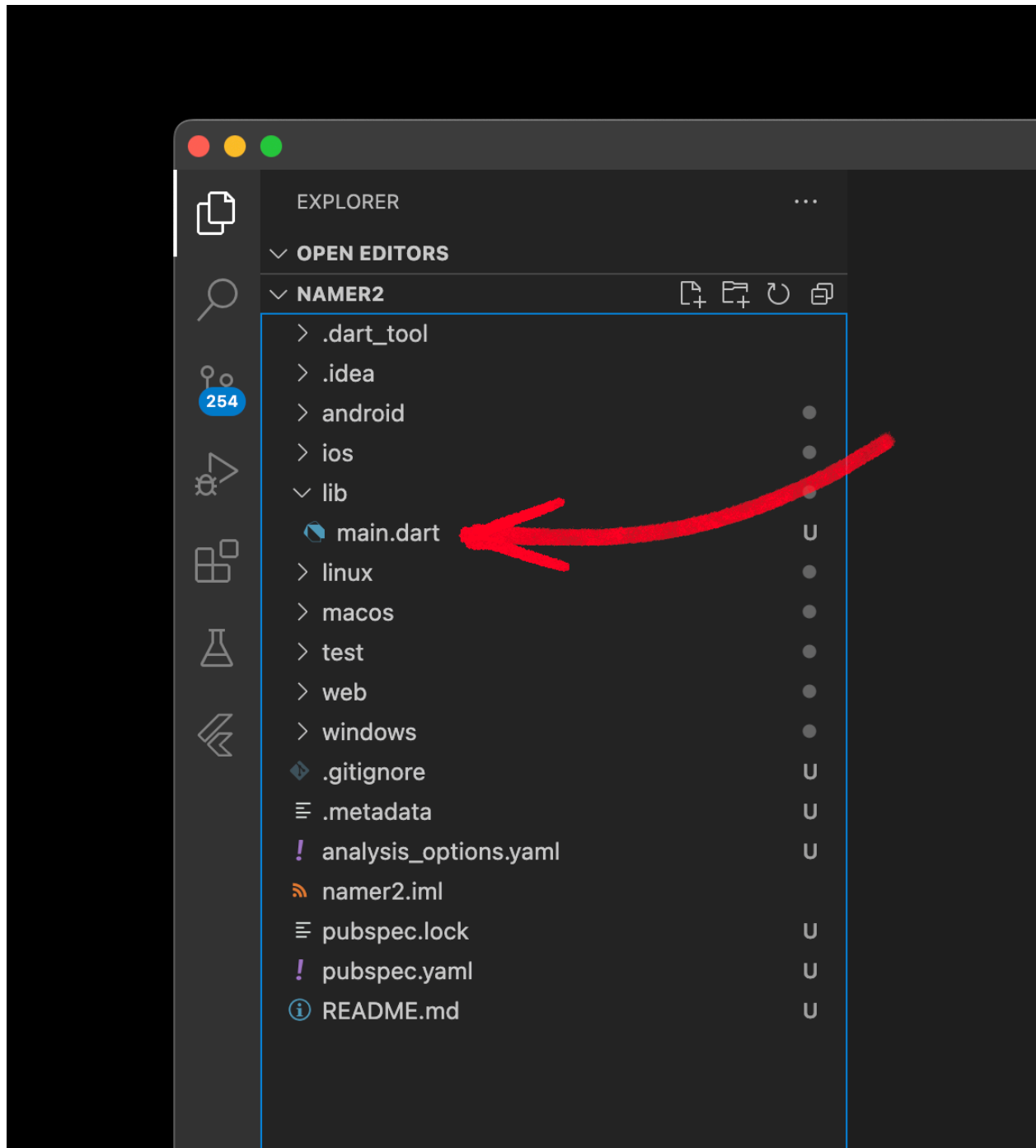
dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^5.0.0

flutter:
  uses-material-design: true
```

`pubspec.yaml` 檔案會指定應用程式的基本資訊，例如目前版本、依附元件，以及隨附的資產。

注意：如果應用程式的名稱不是 `brick_breaker`，請一併變更第一行。

3. 開啟 `lib/` 目錄中的 `main.dart` 檔案。



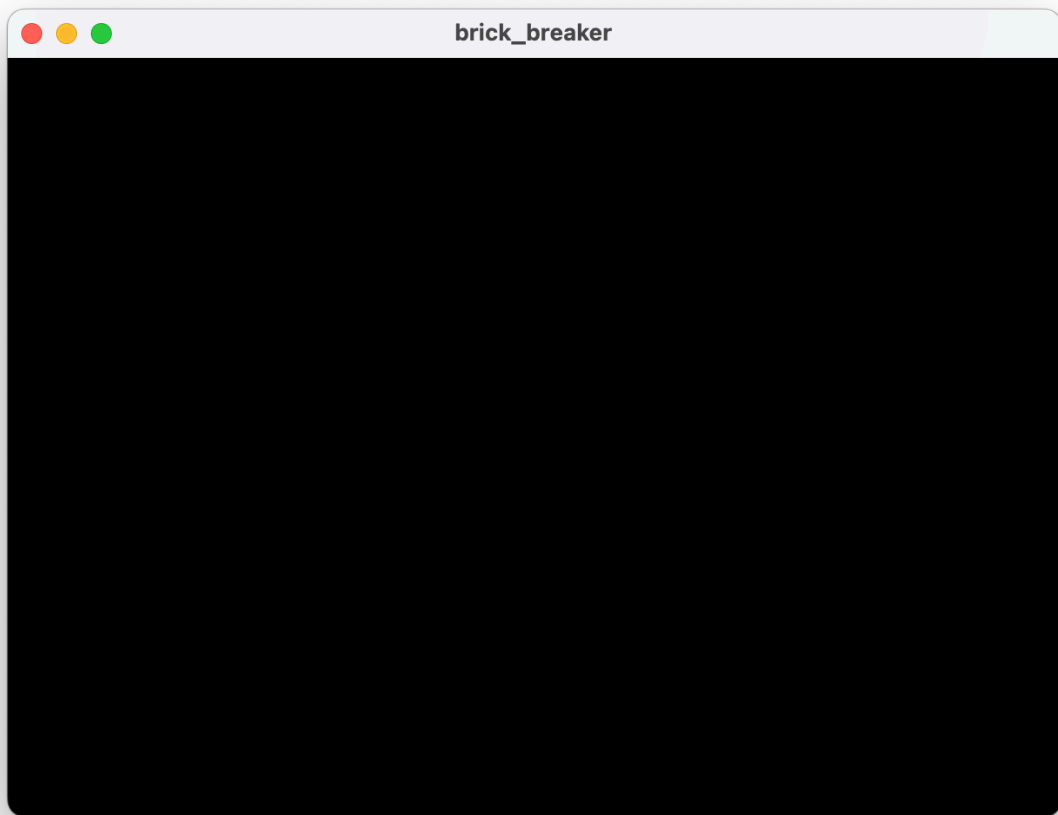
4. 將這個檔案的內容替換成以下內容：

lib/main.dart

```
import 'package:flame/game.dart';
import 'package:flutter/material.dart';

void main() {
  final game = FlameGame();
  runApp(GameWidget(game: game));
}
```

5. 執行這段程式碼，確認一切運作正常。畫面上應該會顯示新視窗，且只有空白的黑色背景。全球最爛的電玩遊戲現在可以 60 FPS 算繪！



4. 建立遊戲

評估遊戲

以 2D 形式進行的遊戲需要遊戲區。您將建構特定維度的區域，然後使用這些維度來調整遊戲其他方面的尺寸。

您可以在遊戲區以各種方式配置座標。根據一項慣例，您可以從畫面中心測量方向，原點 `(0,0)` 位於畫面中心，正值會沿著 x 軸將項目向右移動，沿著 y 軸向上移動。這項標準適用於現今大多數遊戲，尤其是涉及三維空間的遊戲。

當初建立原始 Breakout 遊戲時，慣例是將原點設在左上角。正 x 方向維持不變，但 y 方向已翻轉。x 正向為向右，y 正向為向下。為忠實呈現當時的風格，這款遊戲將原點設為左上角。

附註：不同時代的維度運作方式不同，這可能令人感到困惑。請注意以下兩點：

首先，遊戲工作者的預設值取決於預設情境。對 1970 年代和 1980 年代的程式設計師來說，預設的視覺環境是左上角有 `(0,0)` 的文字緩衝區。這項慣例也沿用至圖形維度的預設值。在 90 年代，3D 遊戲成為主流，當時的程式設計師更著重於三維數學，因此將 `(0,0)` 放在畫面中央更合理。

其次，您可以使用任何一種慣例來表示這些遊戲，只要所有內容一致即可。您可以在螢幕上任何位置放置 `(0,0)`，遊戲玩家不會察覺任何變化。以利於表達問題的方式模擬世界。

在名為 `lib/src` 的新目錄中，建立名為 `config.dart` 的檔案。這個檔案會在後續步驟中加入更多常數。

`lib/src/config.dart`

```
const gameWidth = 820.0;
const gameHeight = 1600.0;
```

這個遊戲的寬度為 820 像素，高度為 1600 像素。遊戲區域會縮放以配合顯示視窗，但新增至畫面的所有元件都會符合這個高度和寬度。

建立 PlayArea

在打磚塊遊戲中，球會從遊戲區的牆壁彈開。如要處理碰撞情形，您必須先使用 `PlayArea` 元件。

1. 在名為 `lib/src/components` 的新目錄中，建立名為 `play_area.dart` 的檔案。
2. 將下列內容加入這個檔案。

`lib/src/components/play_area.dart`

```
import 'dart:async';

import 'package:flame/components.dart';
import 'package:flutter/material.dart';

import '../brick_breaker.dart';

class PlayArea extends RectangleComponent with HasGameReference<BrickBreaker> {
  PlayArea() : super(paint: Paint()..color = const Color(0xffff2e8cf));

  @override
  FutureOr<void> onLoad() async {
    super.onLoad();
    size = Vector2(game.width, game.height);
  }
}
```

注意：輸入上述程式碼後，IDE 會顯示一些紅色的波浪線，您會在稍後建立 `BrickBreaker` 類別時修正這個問題。

Flutter 有 `Widget`，Flame 則有 `Component`。Flutter 應用程式是由小工具樹狀結構組成，而 Flame 遊戲則是由元件樹狀結構組成。

這就是 Flutter 和 Flame 之間有趣的差異。Flutter 的小工具樹狀結構是暫時性說明，用於更新持續性且可變動的 `RenderObject` 層。Flame 的元件是持續且可變動的，開發人員應將這些元件做為模擬系統的一部分。

Flame 的元件經過最佳化，可表達遊戲機制。本程式碼研究室將從遊戲迴圈開始，請見下一個步驟。

3. 為避免雜亂，請新增包含這個專案中所有元件的檔案。在 `lib/src/components` 中建立 `components.dart` 檔案，並加入以下內容。

lib/src/components/components.dart

```
export 'play_area.dart';
```

`export` 指令的作用與 `import` 相反。這個檔案會宣告匯入其他檔案時公開的功能。在後續步驟中新增元件時，這個檔案會增加更多項目。

建立 Flame 遊戲

如要消除上一個步驟中的紅色波浪線，請為 Flame 的 `FlameGame` 衍生新的子類別。

1. 在 `lib/src` 中建立名為 `brick_breaker.dart` 的檔案，並加入下列程式碼。

lib/src/brick_breaker.dart


```
import 'dart:async';

import 'package:flame/components.dart';
import 'package:flame/game.dart';

import 'components/components.dart';
import 'config.dart';

class BrickBreaker extends FlameGame {
  BrickBreaker()
    : super(
        camera: CameraComponent.withFixedResolution(
          width: gameWidth,
          height: gameHeight,
        ),
      );

  double get width => size.x;
  double get height => size.y;

  @override
  FutureOr<void> onLoad() async {
    super.onLoad();

    camera.viewfinder.anchor = Anchor.topLeft;

    world.add(PlayArea());
  }
}
```

這個檔案會協調遊戲的動作。在建構遊戲執行個體期間，這段程式碼會將遊戲設定為使用固定解析度轉譯。遊戲會調整大小，填滿包含遊戲的畫面，並視需要新增[上下黑邊](#)。

您會公開遊戲的寬度和高度，讓 `PlayArea` 等子項元件可自行設定適當大小。

在覆寫的 `onLoad` 方法中，程式碼會執行兩項動作。

1. 將左上角設為觀景窗的錨點。根據預設，`viewfinder` 會使用區域中間做為 `(0,0)` 的錨點。
2. 將 `PlayArea` 新增至 `world`。世界代表遊戲世界。它會透過 `CameraComponent` 的檢視區塊轉換投影所有子項。

警告：您可以直接將 `Component` 新增為 `FlameGame` 的子項。如果這麼做，`CameraComponent` 就不會轉換這些元件，而是將其新增為 `world` 元件的子項。如果視窗大小調整方式與預期不符，您和應用程式的播放器可能會感到困惑。

在螢幕上顯示遊戲畫面

如要查看您在這個步驟中進行的所有變更，請更新 `lib/main.dart` 檔案，並進行下列變更。

lib/main.dart

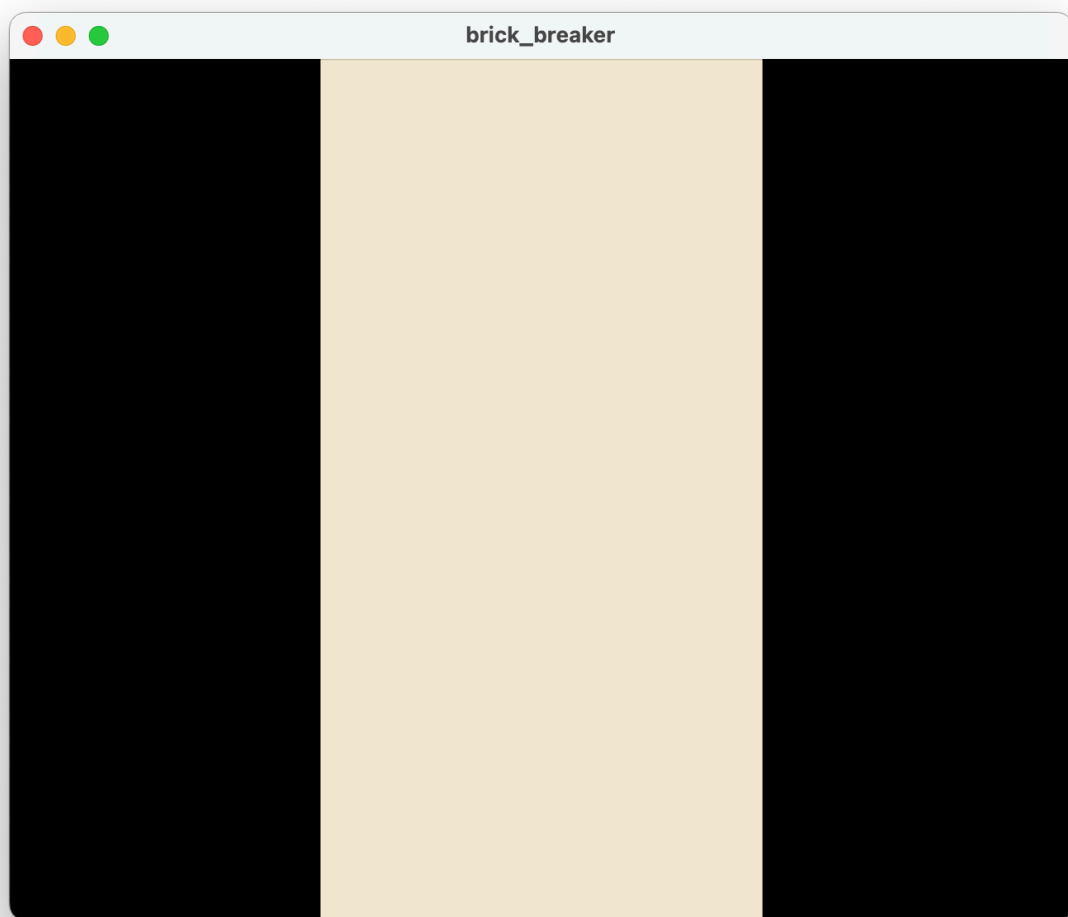
```
import 'package:flame/game.dart';
import 'package:flutter/material.dart';

import 'src/brick_breaker.dart';           // Add this import

void main() {
  final game = BrickBreaker();           // Modify this line
  runApp(GameWidget(game: game));
}
```

完成這些變更後，請重新啟動遊戲。遊戲畫面應如下圖所示。

注意：Flame 不支援 Flutter 的[熱重載](#)功能。您必須徹底重新啟動瀏覽器，變更才會生效。



在下一個步驟中，您將在世界中新增一顆球，並讓球移動！

步驟 5 · 約 5 分鐘

5. 顯示球

建立球體元件

如要在畫面上放置移動中的球，需要建立另一個元件並新增至遊戲世界。

1. 按照下列方式編輯 `lib/src/config.dart` 檔案的內容。

`lib/src/config.dart`

```
const gameWidth = 820.0;
const gameHeight = 1600.0;
const ballRadius = gameWidth * 0.02;                                // Add this constant
```

在本程式碼研究室中，您會多次看到將具名常數定義為衍生值的設計模式。您可以修改頂層的 `gameWidth` 和 `gameHeight`，瞭解遊戲外觀和風格的變化。

2. 在 `lib/src/components` 中，於名為 `ball.dart` 的檔案中建立 `Ball` 元件。

`lib/src/components/ball.dart`

```
import 'package:flame/components.dart';
import 'package:flutter/material.dart';

class Ball extends CircleComponent {
  Ball({
    required this.velocity,
    required super.position,
    required double radius,
  }) : super(
    radius: radius,
    anchor: Anchor.center,
    paint: Paint()
      ..color = const Color(0xff1e6091)
      ..style = PaintingStyle.fill,
  );

  final Vector2 velocity;

  @override
  void update(double dt) {
    super.update(dt);
    position += velocity * dt;
  }
}
```

您先前使用 `RectangleComponent` 定義 `PlayArea`，因此存在更多形狀是合理的。`CircleComponent` (例如 `RectangleComponent`) 衍生自 `PositionedComponent`，因此您可以在畫面上放置球體。更重要的是，可以更新其位置。

這個元件會介紹 `velocity` 的概念，也就是位置隨時間的變化。速度是 `Vector2` 物件，因為速度包含速率和方向。如要更新位置，請覆寫 `update` 方法，遊戲引擎會為每個影格呼叫這個方法。`dt` 是指前一個影格與這個影格之間的時間長度。這有助於您因應不同影格速率 (60 Hz 或 120 Hz) 或因運算量過大而導致影格過長等因素。

附註：在本遊戲中，您會使用向量來模擬具有元件的屬性，例如位置和速度。由於這個遊戲模擬的是 2 維世界，因此位置和速度有兩個分量，分別是 `x` 和 `y`。

將這些元件包裝成單一概念，可簡化向量縮放等概念的處理方式，例如將速度乘以大於 1 的數字，加快球的速度，或是以適當比例的目前速度更新位置。

若要深入瞭解如何使用向量製作模擬，請參閱 Khan Academy 的[向量簡介課程](#)。

請密切注意 `position += velocity * dt` 更新。以下說明如何實作，以便更新一段時間內的動作離散模擬。

3. 如要在元件清單中加入 `Ball` 元件，請按照下列方式編輯 `lib/src/components/components.dart` 檔案。

`lib/src/components/components.dart`

```
export 'ball.dart'; // Add this export
export 'play_area.dart';
```

將球體新增至世界

你有一顆球。將其放置在遊戲世界中，並設定在遊戲區內移動。

按照下列方式編輯 `lib/src/brick_breaker.dart` 檔案。

`lib/src/brick_breaker.dart`

```

import 'dart:async';
import 'dart:math' as math;                                // Add this import

import 'package:flame/components.dart';
import 'package:flame/game.dart';

import 'components/components.dart';
import 'config.dart';

class BrickBreaker extends FlameGame {
  BrickBreaker()
    : super(
        camera: CameraComponent.withFixedResolution(
          width: gameWidth,
          height: gameHeight,
        ),
      );

  final rand = math.Random();                                // Add this variable
  double get width => size.x;
  double get height => size.y;

  @override
  FutureOr<void> onLoad() async {
    super.onLoad();

    camera.viewfinder.anchor = Anchor.topLeft;

    world.add(PlayArea());

    world.add(
      Ball(                                                    // Add from here...
        radius: ballRadius,
        position: size / 2,
        velocity: Vector2(
          (rand.nextDouble() - 0.5) * width,
          height * 0.2,
        ).normalized()..scale(height / 4),
      ),
    );

    debugMode = true;                                         // To here.
  }
}

```

這項變更會將 `Ball` 元件新增至 `world`。如要將球的 `position` 設為顯示區域的中心，程式碼會先將遊戲大小減半，因為 `Vector2` 具有運算子多載（`*` 和 `/`），可依純量值縮放 `Vector2`。

設定球的 `velocity` 涉及更多複雜性。目標是以合理的速度，將球體移到螢幕下方，方向隨機。呼叫 `normalized` 方法會建立 `Vector2` 物件，並將其設為與原始 `Vector2` 相同的方向，但縮放比例會降至 1。無論球往哪個方向移動，速度都會保持一致。接著，球的速度會放大到遊戲高度的 1/4。

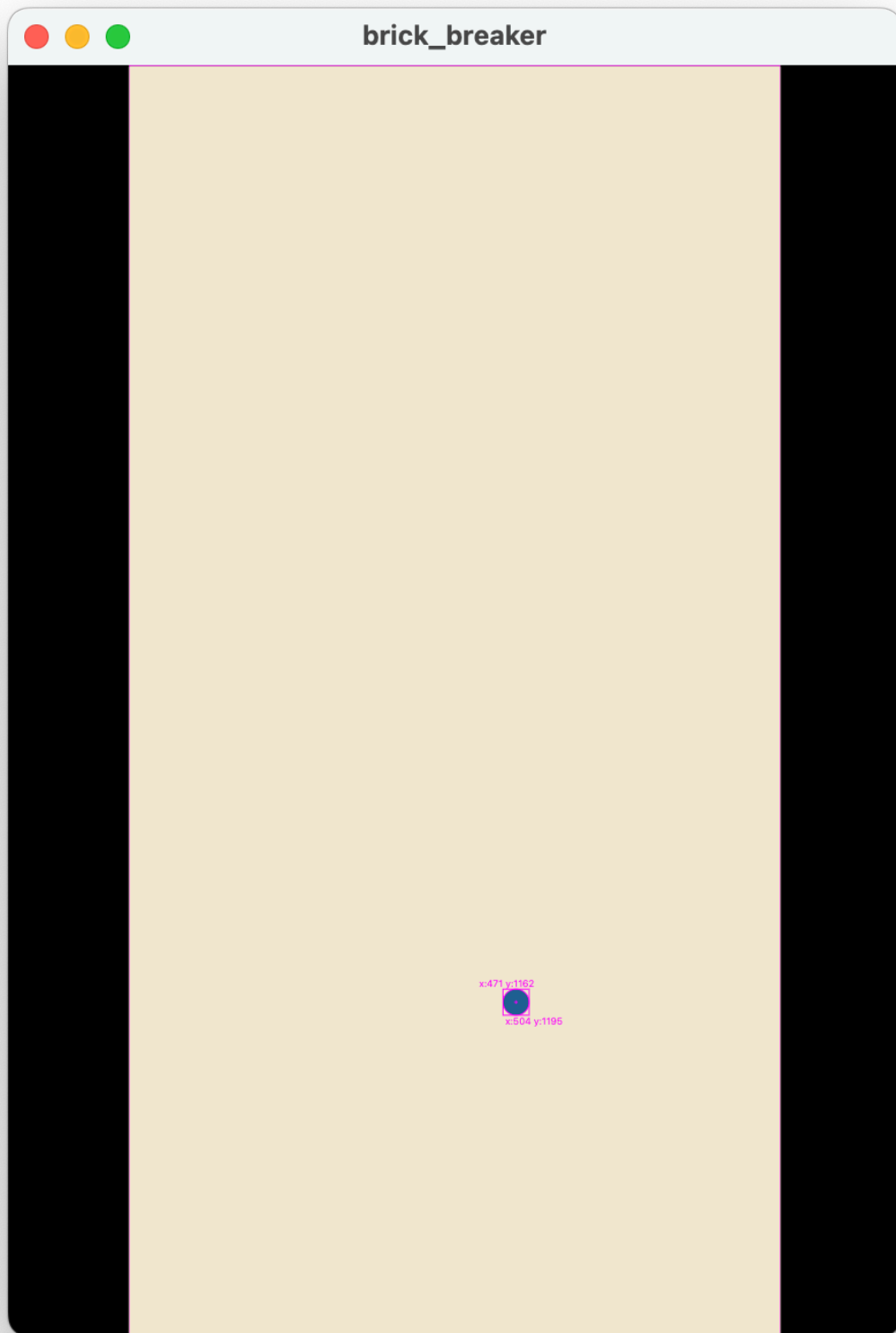
如要取得這些正確值，需要經過幾次疊代，也就是業界所謂的「遊戲測試」。

最後一行會開啟偵錯顯示畫面，在顯示畫面中加入額外資訊，協助您進行偵錯。

附註：向量通常代表方向和大小。大小可能是位置，也可能是速度 (如果是速度的話)。方向是較為一般的概念，且經常需要擷取向量的方向，以便比較兩個向量。

法線是向量的方向，可透過將位置或速度向量轉換為新向量來建立，新向量的方向相同，但大小為 `1`。在先前的程式碼中，正規化用於取得向量或隨機方向，並提供一致的速度。

現在執行遊戲時，畫面應如下所示。



`PlayArea` 和 `Ball` 元件都有偵錯資訊，但背景遮罩會裁剪 `PlayArea` 的數字。之所以會顯示所有項目的偵錯資訊，是因為您已為整個元件樹狀結構開啟 `debugMode`。如果這樣比較有用，您也可以只為所選元件啟用偵錯功能。

如果重新啟動遊戲幾次，您可能會發現球與牆壁的互動不如預期。如要達成這個效果，您需要新增碰撞偵測功能，這會在下一個步驟中完成。

步驟 6 · 約 5 分鐘

6. Bounce around

新增碰撞偵測功能

碰撞偵測會新增行為，讓遊戲辨識兩個物件何時發生接觸。

如要為遊戲新增碰撞偵測功能，請將 `HasCollisionDetection` 混入 `BrickBreaker` 遊戲，如下列程式碼所示。

[lib/src/brick_breaker.dart](#)


```

import 'dart:async';
import 'dart:math' as math;

import 'package:flame/components.dart';
import 'package:flame/game.dart';

import 'components/components.dart';
import 'config.dart';

class BrickBreaker extends FlameGame with HasCollisionDetection { // Modify this line
  BrickBreaker()
    : super(
        camera: CameraComponent.withFixedResolution(
          width: gameWidth,
          height: gameHeight,
        ),
      );

  final rand = math.Random();
  double get width => size.x;
  double get height => size.y;

  @override
  FutureOr<void> onLoad() async {
    super.onLoad();

    camera.viewfinder.anchor = Anchor.topLeft;

    world.add(PlayArea());

    world.add(
      Ball(
        radius: ballRadius,
        position: size / 2,
        velocity: Vector2(
          (rand.nextDouble() - 0.5) * width,
          height * 0.2,
        ).normalized()..scale(height / 4),
      ),
    );

    debugMode = true;
  }
}

```

這會追蹤元件的碰撞箱，並在每個遊戲刻度上觸發碰撞回呼。

附註：碰撞框是碰撞偵測系統用來計算元件是否與其他元件發生碰撞的區域。之所以稱為「碰撞框」，是因為用於碰撞偵測的區域通常會簡化玩家在畫面上看到的視覺呈現內容，通常會簡化為矩形或方塊。為了方便玩家操作，碰撞範圍的大小可能與螢幕上的虛擬人偶不同。

在這個遊戲中，球和磚塊的命中框大小與所附加的元件相同。不過，球棒的碰撞範圍是矩形，沒有螢幕上球棒的圓角。這會讓球棒比實際看起來大一點。

如要開始填入遊戲的命中框，請修改 `PlayArea` 元件，如下所示：

lib/src/components/play_area.dart

```
import 'dart:async';

import 'package:flame/collisions.dart';           // Add this import
import 'package:flame/components.dart';
import 'package:flutter/material.dart';

import '../brick_breaker.dart';

class PlayArea extends RectangleComponent with HasGameReference<BrickBreaker> {
  PlayArea()
    : super(
        paint: Paint()..color = const Color(0xffff2e8cf),
        children: [RectangleHitbox()],              // Add this parameter
      );

  @override
  FutureOr<void> onLoad() async {
    super.onLoad();
    size = Vector2(game.width, game.height);
  }
}
```

將 `RectangleHitbox` 元件新增為 `RectangleComponent` 的子項，即可建構碰撞偵測的點擊方塊，大小與父項元件相符。如果希望熱區小於或大於父項元件，可以使用 `RectangleHitbox` 的工廠建構函式（稱為 `relative`）。

彈跳球

到目前為止，新增碰撞偵測功能對遊戲體驗沒有任何影響。但修改 `Ball` 元件後，系統就會變更該值。當球體與 `PlayArea` 發生碰撞時，必須改變的是球體的行為。

按照下列方式修改 `Ball` 元件。

lib/src/components/ball.dart

```

import 'package:flame/collisions.dart';           // Add this import
import 'package:flame/components.dart';
import 'package:flutter/material.dart';

import '../brick_breaker.dart';                   // And this import
import 'play_area.dart';                         // And this one too

class Ball extends CircleComponent
  with CollisionCallbacks, HasGameReference<BrickBreaker> { // Add these mixins
  Ball({
    required this.velocity,
    required super.position,
    required double radius,
  }) : super(
    radius: radius,
    anchor: Anchor.center,
    paint: Paint()
      ..color = const Color(0xffle6091)
      ..style = PaintingStyle.fill,
    children: [CircleHitbox()], // Add this parameter
  );

  final Vector2 velocity;

  @override
  void update(double dt) {
    super.update(dt);
    position += velocity * dt;
  }

  @override // Add from here...
  void onCollisionStart(
    Set<Vector2> intersectionPoints,
    PositionComponent other,
  ) {
    super.onCollisionStart(intersectionPoints, other);
    if (other is PlayArea) {
      if (intersectionPoints.first.y <= 0) {
        velocity.y = -velocity.y;
      } else if (intersectionPoints.first.x <= 0) {
        velocity.x = -velocity.x;
      } else if (intersectionPoints.first.x >= game.width) {
        velocity.x = -velocity.x;
      } else if (intersectionPoints.first.y >= game.height) {
        removeFromParent();
      }
    } else {
      debugPrint('collision with $other');
    }
  }
} // To here.

```

這個範例新增了 `onCollisionStart` 回呼，因此有重大變更。先前範例中新增至 `BrickBreaker` 的碰撞偵測系統會呼叫這個回呼。

首先，程式碼會測試 `Ball` 是否與 `PlayArea` 發生碰撞。目前遊戲世界中沒有其他元件，因此這似乎是多餘的。在下一個步驟中，您將蝙蝠加入世界時，這項設定就會變更。接著，它還會新增 `else` 條件，處理球與球拍以外的物體碰撞時的情況。溫馨提醒您實作其餘邏輯 (如有)。

當球撞到底牆時，球會從比賽場地消失，但仍可清楚看見。您將在後續步驟中運用 `Flame` 的特效功能處理這項構件。

注意：`Flame` 的碰撞回呼具有生命週期。回呼為 `onCollisionStart`、`onCollision` 和 `onCollisionEnd`。這個遊戲的初始版本是以 `onCollision` 編寫，且需要額外程式碼，才能在後續遊戲時間格處理後續碰撞回呼。針對正確用途使用正確的回呼，可大幅簡化程式碼！

現在球體會與遊戲牆壁發生碰撞，如果能讓玩家使用球拍擊球，一定會很有用...

7. 擊中球

建立 bat 檔案

如要在遊戲中新增球棒，讓球保持在遊戲中，請按照下列步驟操作：

1. 在 `lib/src/config.dart` 檔案中插入一些常數，如下所示。

`lib/src/config.dart`

```
const gameWidth = 820.0;
const gameHeight = 1600.0;
const ballRadius = gameWidth * 0.02;
const batWidth = gameWidth * 0.2; // Add from here...
const batHeight = ballRadius * 2;
const batStep = gameWidth * 0.05; // To here.
```

`batHeight` 和 `batWidth` 常數應該很好理解。另一方面，`batStep` 常數則需要稍微說明。如要與遊戲中的球互動，玩家可以使用滑鼠或手指拖曳球拍（視平台而定），也可以使用鍵盤。`batStep` 常數會設定蝙蝠每次按下向左或向右鍵時的移動距離。

2. 請按照下列方式定義 `Bat` 元件類別。

`lib/src/components/bat.dart`

```

import 'package:flame/collisions.dart';
import 'package:flame/components.dart';
import 'package:flame/effects.dart';
import 'package:flame/events.dart';
import 'package:flutter/material.dart';

import '../brick_breaker.dart';

class Bat extends PositionComponent
  with DragCallbacks, HasGameReference<BrickBreaker> {
  Bat({
    required this.cornerRadius,
    required super.position,
    required super.size,
  }) : super(anchor: Anchor.center, children: [RectangleHitbox()]);

  final Radius cornerRadius;

  final _paint = Paint()
    ..color = const Color(0xff1e6091)
    ..style = PaintingStyle.fill;

  @override
  void render(Canvas canvas) {
    super.render(canvas);
    canvas.drawRRect(
      RRect.fromRectAndRadius(Offset.zero & size.toSize(), cornerRadius),
      _paint,
    );
  }

  @override
  void onDragUpdate(DragUpdateEvent event) {
    super.onDragUpdate(event);
    position.x = (position.x + event.localDelta.x).clamp(0, game.width);
  }

  void moveBy(double dx) {
    add(
      MoveToEffect(
        Vector2((position.x + dx).clamp(0, game.width), position.y),
        EffectController(duration: 0.1),
      ),
    );
  }
}

```

這個元件導入了幾項新功能。

首先，Bat 元件是 `PositionComponent`，不是 `RectangleComponent` 也不是 `CircleComponent`。也就是說，這段程式碼需要在畫面上算繪 Bat。為達成這個目標，它會覆寫 `render` 回呼。

仔細查看 `canvas.drawRRect` (繪製圓角矩形) 呼叫，您可能會問自己：「矩形在哪裡？」 `Offset.zero & size.toSize()` 會利用 `dart:ui` `Offset` 類別的 `operator &` 多載，建立 `Rect`。這種簡寫方式一開始可能會讓您感到困惑，但您會在較低層級的 Flutter 和 Flame 程式碼中經常看到。

其次，這個 `Bat` 元件可使用手指或滑鼠拖曳 (視平台而定)。如要實作這項功能，請新增 `DragCallbacks` 混合，並覆寫 `onDragUpdate` 事件。

最後，`Bat` 元件需要回應鍵盤控制。`moveBy` 函式可讓其他程式碼指示這隻蝙蝠向左或向右移動特定數量的虛擬像素。這項函式會導入 Flame 遊戲引擎的新功能：`Effect`。將 `MoveToEffect` 物件新增為這個元件的子項後，球員就會看到球棒動畫移動到新位置。Flame 提供一系列 `Effect`，可執行各種效果。

`Effect` 的建構函式引數包含對 `game` `getter` 的參照。因此您要在這個類別中加入 `HasGameReference` `mixin`。這個 `mixin` 會將型別安全 `game` 存取子新增至這個元件，以便存取元件樹狀結構頂端的 `BrickBreaker` 執行個體。

您可能會好奇，為何 `Bat` 的 `x` 位置的 `clamp` 允許球棒離開畫面一半。在本程式碼研究室的舊版中，這裡有程式碼可防止球棒離開畫面，但這會降低玩家充分利用球棒彎曲前緣的能力。我們更新了這段程式碼，現在球棒可以滑出螢幕一半，讓遊戲更有趣。

3. 如要讓 `BrickBreaker` 使用 `Bat`，請按照下列方式更新 `lib/src/components/components.dart` 檔案。

lib/src/components/components.dart

```
export 'ball.dart';
export 'bat.dart';                                // Add this export
export 'play_area.dart';
```

將蝙蝠新增至世界

如要將 `Bat` 元件新增至遊戲世界，請更新 `BrickBreaker`，如下所示。

lib/src/brick_breaker.dart

```

import 'dart:async';
import 'dart:math' as math;

import 'package:flame/components.dart';
import 'package:flame/events.dart';           // Add this import
import 'package:flame/game.dart';
import 'package:flutter/material.dart';       // And this import
import 'package:flutter/services.dart';       // And this

import 'components/components.dart';
import 'config.dart';

class BrickBreaker extends FlameGame
  with HasCollisionDetection, KeyboardEvents { // Modify this line
  BrickBreaker()
    : super(
        camera: CameraComponent.withFixedResolution(
          width: gameWidth,
          height: gameHeight,
        ),
      );

  final rand = math.Random();
  double get width => size.x;
  double get height => size.y;

  @override
  FutureOr<void> onLoad() async {
    super.onLoad();

    camera.viewfinder.anchor = Anchor.topLeft;

    world.add(PlayArea());

    world.add(
      Ball(
        radius: ballRadius,
        position: size / 2,
        velocity: Vector2(
          (rand.nextDouble() - 0.5) * width,
          height * 0.2,
        ).normalized()..scale(height / 4),
      ),
    );

    world.add( // Add from here...
      Bat(
        size: Vector2(batWidth, batHeight),
        cornerRadius: const Radius.circular(ballRadius / 2),
        position: Vector2(width / 2, height * 0.95),
      ),
    ); // To here.
  }

```



```

    debugMode = true;
  }

  @override // Add from here...
  KeyEventResult onKeyEvent(
    KeyEvent event,
    Set<LogicalKeyboardKey> keysPressed,
  ) {
    super.onKeyEvent(event, keysPressed);
    switch (event.logicalKey) {
      case LogicalKeyboardKey.arrowLeft:
        world.children.query<Bat>().first.moveBy(-batStep);
      case LogicalKeyboardKey.arrowRight:
        world.children.query<Bat>().first.moveBy(batStep);
    }
    return KeyEventResult.handled;
  } // To here.
}

```

新增的 `KeyboardEvents` mixin 和覆寫的 `onKeyEvent` 方法會處理鍵盤輸入。回想您先前新增的程式碼，將球棒移動適當的步數。

其餘新增的程式碼區塊會將蝙蝠加入遊戲世界，並放在適當位置，比例也正確。這個檔案會顯示所有這些設定，方便您調整球棒和球的相對大小，找出最適合的遊戲感覺。

如果此時玩遊戲，你會發現可以移動球棒攔截球，但除了留在 `Ball` 碰撞偵測程式碼中的偵錯記錄外，沒有任何可見的回應。

現在就來修正這個問題。按照下列方式編輯 `Ball` 元件。

[lib/src/components/ball.dart](#)

```

import 'package:flame/collisions.dart';
import 'package:flame/components.dart';
import 'package:flame/effects.dart';           // Add this import
import 'package:flutter/material.dart';

import '../brick_breaker.dart';
import 'bat.dart';                             // And this import
import 'play_area.dart';

class Ball extends CircleComponent
  with CollisionCallbacks, HasGameReference<BrickBreaker> {
  Ball({
    required this.velocity,
    required super.position,
    required double radius,
  }) : super(
    radius: radius,
    anchor: Anchor.center,
    paint: Paint()
      ..color = const Color(0xff1e6091)
      ..style = PaintingStyle.fill,
    children: [CircleHitbox()],
  );

  final Vector2 velocity;

  @override
  void update(double dt) {
    super.update(dt);
    position += velocity * dt;
  }

  @override
  void onCollisionStart(
    Set<Vector2> intersectionPoints,
    PositionComponent other,
  ) {
    super.onCollisionStart(intersectionPoints, other);
    if (other is PlayArea) {
      if (intersectionPoints.first.y <= 0) {
        velocity.y = -velocity.y;
      } else if (intersectionPoints.first.x <= 0) {
        velocity.x = -velocity.x;
      } else if (intersectionPoints.first.x >= game.width) {
        velocity.x = -velocity.x;
      } else if (intersectionPoints.first.y >= game.height) {
        add(RemoveEffect(delay: 0.35)); // Modify from here...
      }
    } else if (other is Bat) {
      velocity.y = -velocity.y;
      velocity.x =
        velocity.x +

```

```
        (position.x - other.position.x) / other.size.x * game.width * 0.3;
    } else {
        debugPrint('collision with $other');
    }
}
}
```

這些程式碼變更可修正兩個不同的問題。

首先，修正了球體在觸及螢幕底部時消失的問題。如要修正這個問題，請將 `removeFromParent` 呼叫替換為 `RemoveEffect`。 `RemoveEffect` 會在球離開可檢視的遊戲區域後，將球從遊戲世界中移除。

其次，這些變更修正了球棒和球之間發生碰撞的處理方式。這段處理程式碼對玩家非常有利。只要球員用球棒觸碰球，球就會回到畫面頂端。如果覺得這樣太寬鬆，想要更貼近現實的體驗，請變更這項操控設定，讓遊戲更符合您的期望。

值得一提的是，`velocity` 更新相當複雜。這不只是反轉速度的 `y` 分量，就像牆壁碰撞一樣。此外，它也會更新 `x` 元件，更新方式取決於球棒和球接觸時的相對位置。這讓玩家能進一步掌控球的動作，但除了實際操作，遊戲不會以任何方式向玩家說明具體做法。

現在您有了球拍可以擊球，如果能用球擊破磚塊，那就太棒了！

8. 拆除牆壁

建立積木

如要在遊戲中新增積木，請按照下列步驟操作：

1. 在 `lib/src/config.dart` 檔案中插入一些常數，如下所示。

`lib/src/config.dart`

```
import 'package:flutter/material.dart';                                // Add this import

const brickColors = [                                                // Add this const
  Color(0xffff94144),
  Color(0xffff3722c),
  Color(0xffff8961e),
  Color(0xffff9844a),
  Color(0xffff9c74f),
  Color(0xff90be6d),
  Color(0xff43aa8b),
  Color(0xff4d908e),
  Color(0xff277da1),
  Color(0xff577590),
];

const gameWidth = 820.0;
const gameHeight = 1600.0;
const ballRadius = gameWidth * 0.02;
const batWidth = gameWidth * 0.2;
const batHeight = ballRadius * 2;
const batStep = gameWidth * 0.05;
const brickGutter = gameWidth * 0.015;                                // Add from here...
final brickWidth =
  (gameWidth - (brickGutter * (brickColors.length + 1))) / brickColors.length;
const brickHeight = gameHeight * 0.03;
const difficultyModifier = 1.03;                                      // To here.
```

2. 插入 `Brick` 元件，如下所示。

`lib/src/components/brick.dart`

```

import 'package:flame/collisions.dart';
import 'package:flame/components.dart';
import 'package:flutter/material.dart';

import '../brick_breaker.dart';
import '../config.dart';
import 'ball.dart';
import 'bat.dart';

class Brick extends RectangleComponent
  with CollisionCallbacks, HasGameReference<BrickBreaker> {
  Brick({required super.position, required Color color})
    : super(
        size: Vector2(brickWidth, brickHeight),
        anchor: Anchor.center,
        paint: Paint()
          ..color = color
          ..style = PaintingStyle.fill,
        children: [RectangleHitbox()],
      );

  @override
  void onCollisionStart(
    Set<Vector2> intersectionPoints,
    PositionComponent other,
  ) {
    super.onCollisionStart(intersectionPoints, other);
    removeFromParent();

    if (game.world.children.query<Brick>().length == 1) {
      game.world.removeAll(game.world.children.query<Ball>());
      game.world.removeAll(game.world.children.query<Bat>());
    }
  }
}

```

到目前為止，您應該很熟悉大部分的程式碼。這段程式碼使用 `RectangleComponent`，並在元件樹狀結構頂端提供碰撞偵測和 `BrickBreaker` 遊戲的型別安全參照。

這段程式碼導入最重要的全新概念，就是玩家如何達成勝利條件。勝出條件檢查會查詢世界上的積木，並確認只剩一個。這可能有點令人困惑，因為前一行會從父項移除這個磚塊。

請務必瞭解，移除元件是排入佇列的指令。這個程式碼會在執行後移除積木，但會在遊戲世界下一個刻度之前移除。

如要讓 `Brick` 元件可供 `BrickBreaker` 存取，請按照下列方式編輯 `lib/src/components/components.dart`。

[lib/src/components/components.dart](#)

```
export 'ball.dart';  
export 'bat.dart';  
export 'brick.dart';  
export 'play_area.dart';
```

```
// Add this export
```

在世界中新增積木

將 `Ball` 元件更新為下列內容。

[lib/src/components/ball.dart](#)

```

import 'package:flame/collisions.dart';
import 'package:flame/components.dart';
import 'package:flame/effects.dart';
import 'package:flutter/material.dart';

import '../brick_breaker.dart';
import 'bat.dart';
import 'brick.dart'; // Add this import
import 'play_area.dart';

class Ball extends CircleComponent
  with CollisionCallbacks, HasGameReference<BrickBreaker> {
  Ball({
    required this.velocity,
    required super.position,
    required double radius,
    required this.difficultyModifier, // Add this parameter
  }) : super(
    radius: radius,
    anchor: Anchor.center,
    paint: Paint()
      ..color = const Color(0xfffe6091)
      ..style = PaintingStyle.fill,
    children: [CircleHitbox()],
  );

  final Vector2 velocity;
  final double difficultyModifier; // Add this member

  @override
  void update(double dt) {
    super.update(dt);
    position += velocity * dt;
  }

  @override
  void onCollisionStart(
    Set<Vector2> intersectionPoints,
    PositionComponent other,
  ) {
    super.onCollisionStart(intersectionPoints, other);
    if (other is PlayArea) {
      if (intersectionPoints.first.y <= 0) {
        velocity.y = -velocity.y;
      } else if (intersectionPoints.first.x <= 0) {
        velocity.x = -velocity.x;
      } else if (intersectionPoints.first.x >= game.width) {
        velocity.x = -velocity.x;
      } else if (intersectionPoints.first.y >= game.height) {
        add(RemoveEffect(delay: 0.35));
      }
    } else if (other is Bat) {

```

```

    velocity.y = -velocity.y;
    velocity.x =
        velocity.x +
        (position.x - other.position.x) / other.size.x * game.width * 0.3;
} else if (other is Brick) { // Modify from here...
    if (position.y < other.position.y - other.size.y / 2) {
        velocity.y = -velocity.y;
    } else if (position.y > other.position.y + other.size.y / 2) {
        velocity.y = -velocity.y;
    } else if (position.x < other.position.x) {
        velocity.x = -velocity.x;
    } else if (position.x > other.position.x) {
        velocity.x = -velocity.x;
    }
    velocity.setFrom(velocity * difficultyModifier); // To here.
}
}
}

```

這項功能只會導入一個新元素，也就是難度修飾符，每次撞到磚塊後都會增加球速。您需要進行遊戲測試，找出適合自家遊戲的適當難度曲線，才能調整這個參數。

按照下列方式編輯 `BrickBreaker` 遊戲。

[lib/src/brick_breaker.dart](#)


```

import 'dart:async';
import 'dart:math' as math;

import 'package:flame/components.dart';
import 'package:flame/events.dart';
import 'package:flame/game.dart';
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';

import 'components/components.dart';
import 'config.dart';

class BrickBreaker extends FlameGame
  with HasCollisionDetection, KeyboardEvents {
  BrickBreaker()
    : super(
        camera: CameraComponent.withFixedResolution(
          width: gameWidth,
          height: gameHeight,
        ),
      );

  final rand = math.Random();
  double get width => size.x;
  double get height => size.y;

  @override
  FutureOr<void> onLoad() async {
    super.onLoad();

    camera.viewfinder.anchor = Anchor.topLeft;

    world.add(PlayArea());

    world.add(
      Ball(
        difficultyModifier: difficultyModifier, // Add this argument
        radius: ballRadius,
        position: size / 2,
        velocity: Vector2(
          (rand.nextDouble() - 0.5) * width,
          height * 0.2,
        ).normalized()..scale(height / 4),
      ),
    );

    world.add(
      Bat(
        size: Vector2(batWidth, batHeight),
        cornerRadius: const Radius.circular(ballRadius / 2),
        position: Vector2(width / 2, height * 0.95),
      ),
    );
  }

```

```

);

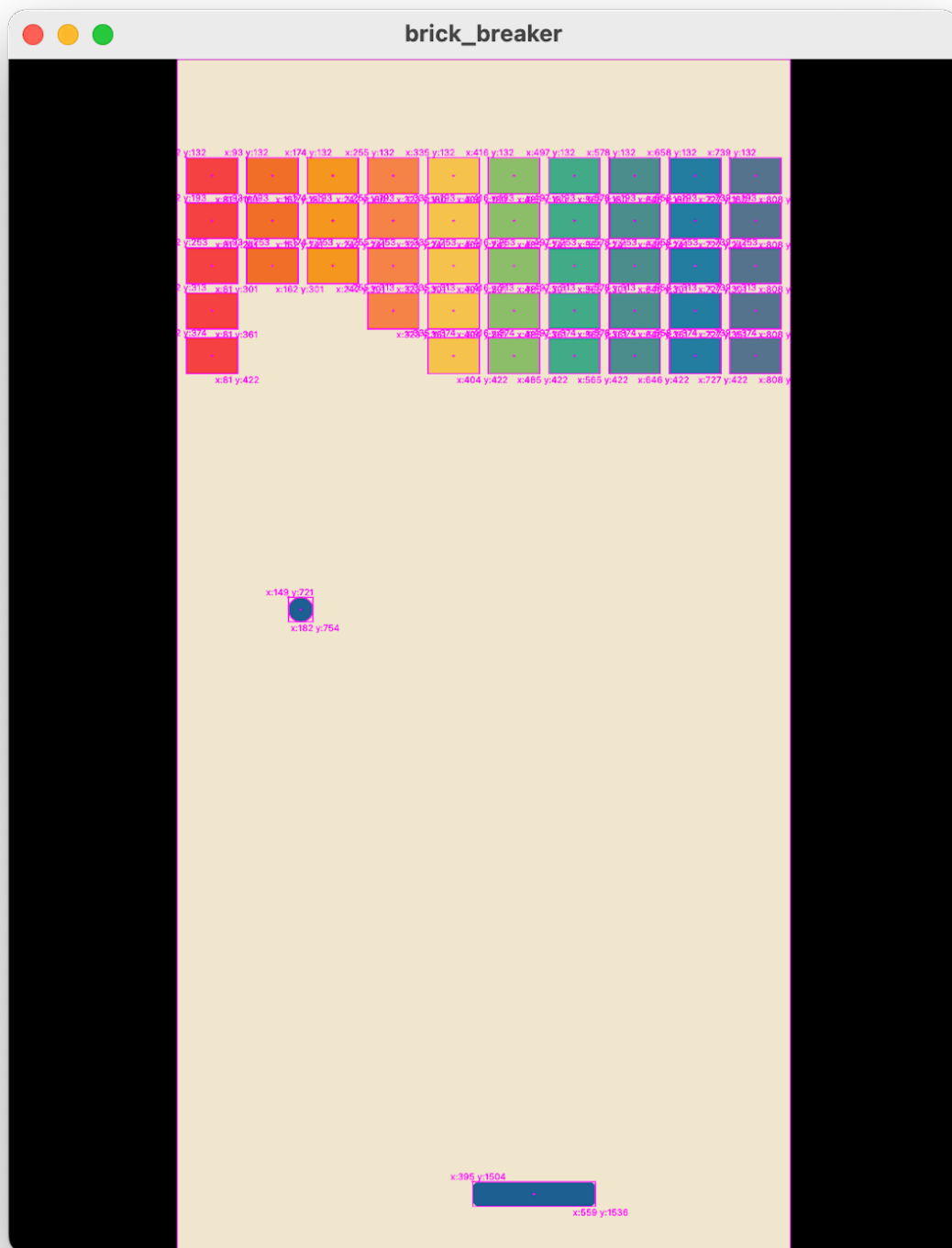
await world.addAll([                                     // Add from here...
  for (var i = 0; i < brickColors.length; i++)
    for (var j = 1; j <= 5; j++)
      Brick(
        position: Vector2(
          (i + 0.5) * brickWidth + (i + 1) * brickGutter,
          (j + 2.0) * brickHeight + j * brickGutter,
        ),
        color: brickColors[i],
      ),
]);                                                       // To here.

debugMode = true;
}

@override
KeyEventResult onKeyEvent(
  KeyEvent event,
  Set<LogicalKeyboardKey> keysPressed,
) {
  super.onKeyEvent(event, keysPressed);
  switch (event.logicalKey) {
    case LogicalKeyboardKey.arrowLeft:
      world.children.query<Bat>().first.moveBy(-batStep);
    case LogicalKeyboardKey.arrowRight:
      world.children.query<Bat>().first.moveBy(batStep);
  }
  return KeyEventResult.handled;
}
}

```

執行遊戲時，系統會顯示所有重要遊戲機制。您可以關閉偵錯功能並視為完成，但總覺得少了些什麼。



歡迎畫面、遊戲結束畫面和分數如何？Flutter 可在遊戲中新增這些功能，這也是您接下來要關注的重點。

步驟 9 · 約 10 分鐘

9. 贏得遊戲

新增播放狀態

在這個步驟中，您會在 Flutter 包裝函式中嵌入 Flame 遊戲，然後為歡迎、遊戲結束和獲勝畫面新增 Flutter 疊加層。

首先，您要修改遊戲和元件檔案，實作可反映是否顯示疊加層的播放狀態，以及要顯示哪個疊加層。

1. 按照下列方式修改 `BrickBreaker` 遊戲。

[lib/src/brick_breaker.dart](#)

```

import 'dart:async';
import 'dart:math' as math;

import 'package:flame/components.dart';
import 'package:flame/events.dart';
import 'package:flame/game.dart';
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';

import 'components/components.dart';
import 'config.dart';

enum PlayState { welcome, playing, gameOver, won } // Add this enumeration

class BrickBreaker extends FlameGame
  with HasCollisionDetection, KeyboardEvents, TapDetector { // Modify this line
  BrickBreaker()
    : super(
        camera: CameraComponent.withFixedResolution(
          width: gameWidth,
          height: gameHeight,
        ),
      );

  final rand = math.Random();
  double get width => size.x;
  double get height => size.y;

  late PlayState _playState; // Add from here...
  PlayState get playState => _playState;
  set playState(PlayState playState) {
    _playState = playState;
    switch (playState) {
      case PlayState.welcome:
      case PlayState.gameOver:
      case PlayState.won:
        overlays.add(playState.name);
      case PlayState.playing:
        overlays.remove(PlayState.welcome.name);
        overlays.remove(PlayState.gameOver.name);
        overlays.remove(PlayState.won.name);
    }
  } // To here.

  @override
  FutureOr<void> onLoad() async {
    super.onLoad();

    camera.viewfinder.anchor = Anchor.topLeft;

    world.add(PlayArea());
  }
}

```

```

playState = PlayState.welcome; // Add from here...
}

void startGame() {
    if (playState == PlayState.playing) return;

    world.removeAll(world.children.query<Ball>());
    world.removeAll(world.children.query<Bat>());
    world.removeAll(world.children.query<Brick>());

    playState = PlayState.playing; // To here.

    world.add(
        Ball(
            difficultyModifier: difficultyModifier,
            radius: ballRadius,
            position: size / 2,
            velocity: Vector2(
                (rand.nextDouble() - 0.5) * width,
                height * 0.2,
            ).normalized()..scale(height / 4),
        ),
    );

    world.add(
        Bat(
            size: Vector2(batWidth, batHeight),
            cornerRadius: const Radius.circular(ballRadius / 2),
            position: Vector2(width / 2, height * 0.95),
        ),
    );

    world.addAll([ // Drop the await
        for (var i = 0; i < brickColors.length; i++)
            for (var j = 1; j <= 5; j++)
                Brick(
                    position: Vector2(
                        (i + 0.5) * brickWidth + (i + 1) * brickGutter,
                        (j + 2.0) * brickHeight + j * brickGutter,
                    ),
                    color: brickColors[i],
                ),
    ]); // Drop the debugMode

}

@override // Add from here...
void onTap() {
    super.onTap();
    startGame();
} // To here.

@override

```

```

KeyEventResult onKeyEvent(
  KeyEvent event,
  Set<LogicalKeyboardKey> keysPressed,
) {
  super.onKeyEvent(event, keysPressed);
  switch (event.logicalKey) {
    case LogicalKeyboardKey.arrowLeft:
      world.children.query<Bat>().first.moveBy(-batStep);
    case LogicalKeyboardKey.arrowRight:
      world.children.query<Bat>().first.moveBy(batStep);
    case LogicalKeyboardKey.space: // Add from here...
    case LogicalKeyboardKey.enter:
      startGame(); // To here.
  }
  return KeyEventResult.handled;
}

@override
Color backgroundColor() => const Color(0xffff2e8cf); // Add this override
}

```

這段程式碼會大幅變更 `BrickBreaker` 遊戲。新增 `playState` 列舉需要大量工作。這會記錄玩家進入、進行遊戲，以及輸贏的過程。在檔案頂端定義列舉，然後將其列項化為隱藏狀態，並搭配相符的 `getter` 和 `setter`。當遊戲的各個部分觸發播放狀態轉換時，這些 `getter` 和 `setter` 可啟用疊加層的修改作業。

接著，您會將 `onLoad` 中的程式碼分割為 `onLoad` 和新的 `startGame` 方法。在此之前，您只能透過重新啟動遊戲來開始新遊戲。有了這些新功能，玩家現在不必採取如此激烈的措施，就能開始新遊戲。

您為遊戲設定了兩個新的處理常式，允許玩家開始新遊戲。您新增了輕觸事件處理常式，並擴充鍵盤事件處理常式，讓使用者能以多種模式開始新遊戲。建立播放狀態模型後，當玩家獲勝或輸掉時，更新元件以觸發播放狀態轉換就很有意義。

2. 按照下列方式修改 `Ball` 元件。

<lib/src/components/ball.dart>

```

import 'package:flame/collisions.dart';
import 'package:flame/components.dart';
import 'package:flame/effects.dart';
import 'package:flutter/material.dart';

import '../brick_breaker.dart';
import 'bat.dart';
import 'brick.dart';
import 'play_area.dart';

class Ball extends CircleComponent
  with CollisionCallbacks, HasGameReference<BrickBreaker> {
  Ball({
    required this.velocity,
    required super.position,
    required double radius,
    required this.difficultyModifier,
  }) : super(
    radius: radius,
    anchor: Anchor.center,
    paint: Paint()
      ..color = const Color(0xff1e6091)
      ..style = PaintingStyle.fill,
    children: [CircleHitbox()],
  );

  final Vector2 velocity;
  final double difficultyModifier;

  @override
  void update(double dt) {
    super.update(dt);
    position += velocity * dt;
  }

  @override
  void onCollisionStart(
    Set<Vector2> intersectionPoints,
    PositionComponent other,
  ) {
    super.onCollisionStart(intersectionPoints, other);
    if (other is PlayArea) {
      if (intersectionPoints.first.y <= 0) {
        velocity.y = -velocity.y;
      } else if (intersectionPoints.first.x <= 0) {
        velocity.x = -velocity.x;
      } else if (intersectionPoints.first.x >= game.width) {
        velocity.x = -velocity.x;
      } else if (intersectionPoints.first.y >= game.height) {
        add(
          RemoveEffect(
            delay: 0.35,

```



```

        onComplete: () {
            game.playState = PlayState.gameOver;
        },
    ),
);
}
} else if (other is Bat) {
    velocity.y = -velocity.y;
    velocity.x =
        velocity.x +
        (position.x - other.position.x) / other.size.x * game.width * 0.3;
} else if (other is Brick) {
    if (position.y < other.position.y - other.size.y / 2) {
        velocity.y = -velocity.y;
    } else if (position.y > other.position.y + other.size.y / 2) {
        velocity.y = -velocity.y;
    } else if (position.x < other.position.x) {
        velocity.x = -velocity.x;
    } else if (position.x > other.position.x) {
        velocity.x = -velocity.x;
    }
    velocity.setFrom(velocity * difficultyModifier);
}
}
}

```

這項小變更會在 `RemoveEffect` 中新增 `onComplete` 回呼，觸發 `gameOver` 播放狀態。如果播放器允許球從畫面底部脫離，這應該會感覺差不多。

3. 按照下列方式編輯 `Brick` 元件。

<lib/src/components/brick.dart>

```

import 'package:flame/collisions.dart';
import 'package:flame/components.dart';
import 'package:flutter/material.dart';

import '../brick_breaker.dart';
import '../config.dart';
import 'ball.dart';
import 'bat.dart';

class Brick extends RectangleComponent
  with CollisionCallbacks, HasGameReference<BrickBreaker> {
  Brick({required super.position, required Color color})
    : super(
        size: Vector2(brickWidth, brickHeight),
        anchor: Anchor.center,
        paint: Paint()
          ..color = color
          ..style = PaintingStyle.fill,
        children: [RectangleHitbox()],
      );

  @override
  void onCollisionStart(
    Set<Vector2> intersectionPoints,
    PositionComponent other,
  ) {
    super.onCollisionStart(intersectionPoints, other);
    removeFromParent();

    if (game.world.children.query<Brick>().length == 1) {
      game.playState = PlayState.won; // Add this line
      game.world.removeAll(game.world.children.query<Ball>());
      game.world.removeAll(game.world.children.query<Bat>());
    }
  }
}

```

反之，如果玩家能擊碎所有磚塊，就會看到「遊戲獲勝」畫面。做得好，玩家！

新增 Flutter 包裝函式

如要提供遊戲的嵌入位置，並新增播放狀態疊加層，請新增 Flutter Shell。

1. 在 `lib/src` 下建立 `widgets` 目錄。
2. 新增 `game_app.dart` 檔案，並在該檔案中插入下列內容。

[lib/src/widgets/game_app.dart](#)

```

import 'package:flame/game.dart';
import 'package:flutter/material.dart';
import 'package:google_fonts/google_fonts.dart';

import '../brick_breaker.dart';
import '../config.dart';

class GameApp extends StatelessWidget {
  const GameApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        textTheme: GoogleFonts.pressStart2pTextTheme().apply(
          bodyColor: const Color(0xff184e77),
          displayColor: const Color(0xff184e77),
        ),
      ),
      home: Scaffold(
        body: Container(
          decoration: const BoxDecoration(
            gradient: LinearGradient(
              begin: Alignment.topCenter,
              end: Alignment.bottomCenter,
              colors: [Color(0xffa9d6e5), Color(0xffff2e8cf)],
            ),
          ),
          child: SafeArea(
            child: Padding(
              padding: const EdgeInsets.all(16),
              child: Center(
                child: FittedBox(
                  child: SizedBox(
                    width: gameWidth,
                    height: gameHeight,
                    child: GameWidget.controlled(
                      gameFactory: BrickBreaker.new,
                      overlayBuilderMap: {
                        PlayState.welcome.name: (context, game) => Center(
                          child: Text(
                            'TAP TO PLAY',
                            style: Theme.of(context).textTheme.headlineLarge,
                          ),
                        ),
                        PlayState.gameOver.name: (context, game) => Center(
                          child: Text(
                            'G A M E   O V E R',
                            style: Theme.of(context).textTheme.headlineLarge,
                          ),
                        ),
                      },
                    ),
                  ),
                ),
              ),
            ),
          ),
        ),
      ),
    );
  }
}

```

```
        PlayState.won.name: (context, game) => Center(
            child: Text(
                'Y O U   W O N ! ! !',
                style: Theme.of(context).textTheme.headlineLarge,
            ),
        ),
    ),
},
),
),
),
),
),
),
),
),
),
);
}
```

這個檔案中的大部分內容都遵循標準的 Flutter 小工具樹狀結構建構方式。Flame 專屬的部分包括使用

這個 `overlayBuilderMap` 的鍵必須與 `BrickBreaker` 中的 `playState` 設定器新增或移除的疊加層一致。如果嘗試設定不在這張地圖中的疊加層，就會導致所有臉部表情都變成不开心。

3. 如要在畫面上顯示這項新功能，請將 `lib/main.dart` 檔案替換為下列內容。

lib/main.dart

```
import 'package:flutter/material.dart';

import 'src/widgets/game_app.dart';

void main() {
  runApp(const GameApp());
}
```

如果您在 iOS、Linux、Windows 或網頁上執行這段程式碼，遊戲中會顯示預期輸出內容。如果您以 macOS 或 Android 為目標平台，還需要進行最後一項調整，才能啟用 `google fonts` 顯示功能。

啟用字型存取權

新增 Android 的網際網路權限

如果是 Android，您必須新增網際網路權限。請按照下列方式編輯 `AndroidManifest.xml`。

android/app/src/main/AndroidManifest.xml

```

<manifest xmlns:android="http://schemas.android.com/apk/res/android">
  <!-- Add the following line -->
  <uses-permission android:name="android.permission.INTERNET" />
  <application
    android:label="brick_breaker"
    android:name="{applicationName}"
    android:icon="@mipmap/ic_launcher">
    <activity
      android:name=".MainActivity"
      android:exported="true"
      android:launchMode="singleTop"
      android:taskAffinity=""
      android:theme="@style/LaunchTheme"

      android:configChanges="orientation|keyboardHidden|keyboard|screenSize|smallestScreenSize|loca
le|layoutDirection|fontScale|screenLayout|density|uiMode"
      android:hardwareAccelerated="true"
      android:windowSoftInputMode="adjustResize">
      <!-- Specifies an Android theme to apply to this Activity as soon as
        the Android process has started. This theme is visible to the user
        while the Flutter UI initializes. After that, this theme continues
        to determine the Window background behind the Flutter UI. -->
      <meta-data
        android:name="io.flutter.embedding.android.NormalTheme"
        android:resource="@style/NormalTheme"
      />
      <intent-filter>
        <action android:name="android.intent.action.MAIN"/>
        <category android:name="android.intent.category.LAUNCHER"/>
      </intent-filter>
    </activity>
    <!-- Don't delete the meta-data below.
      This is used by the Flutter tool to generate GeneratedPluginRegistrant.java -->
    <meta-data
      android:name="flutterEmbedding"
      android:value="2" />
  </application>
  <!-- Required to query activities that can process text, see:
    https://developer.android.com/training/package-visibility and
    https://developer.android.com/reference/android/content/Intent#ACTION_PROCESS_TEXT.

    In particular, this is used by the Flutter engine in
    io.flutter.plugin.text.ProcessTextPlugin. -->
  <queries>
    <intent>
      <action android:name="android.intent.action.PROCESS_TEXT"/>
      <data android:mimeType="text/plain"/>
    </intent>
  </queries>
</manifest>

```

編輯 macOS 的授權檔案

在 macOS 中，您需要編輯兩個檔案。

1. 編輯 `DebugProfile.entitlements` 檔案，使其符合下列程式碼。

macos/Runner/DebugProfile.entitlements

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
    <key>com.apple.security.app-sandbox</key>
    <true/>
    <key>com.apple.security.cs.allow-jit</key>
    <true/>
    <key>com.apple.security.network.server</key>
    <true/>
    <!-- Add from here... -->
    <key>com.apple.security.network.client</key>
    <true/>
    <!-- to here. -->
</dict>
</plist>
```

2. 編輯 `Release.entitlements` 檔案，使其符合下列程式碼

macos/Runner/Release.entitlements

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
<dict>
    <key>com.apple.security.app-sandbox</key>
    <true/>
    <!-- Add from here... -->
    <key>com.apple.security.network.client</key>
    <true/>
    <!-- to here. -->
</dict>
</plist>
```

直接執行這個程式碼，應該會在所有平台上顯示歡迎畫面，以及遊戲結束或獲勝畫面。這些畫面可能有點簡單，如果能顯示分數就更好了。因此，請猜猜您會在下一個步驟中做什麼！

步驟 10 · 約 5 分鐘

10. 計算分數

為遊戲新增分數

在這個步驟中，您會將遊戲分數公開給周圍的 Flutter 內容。在這個步驟中，您會將 Flame 遊戲的狀態公開給周圍的 Flutter 狀態管理。這樣一來，遊戲程式碼就能在玩家每打破一塊磚頭時更新分數。

1. 按照下列方式修改 `BrickBreaker` 遊戲。

[lib/src/brick_breaker.dart](#)

```

import 'dart:async';
import 'dart:math' as math;

import 'package:flame/components.dart';
import 'package:flame/events.dart';
import 'package:flame/game.dart';
import 'package:flutter/material.dart';
import 'package:flutter/services.dart';

import 'components/components.dart';
import 'config.dart';

enum PlayState { welcome, playing, gameOver, won }

class BrickBreaker extends FlameGame
  with HasCollisionDetection, KeyboardEvents, TapDetector {
  BrickBreaker()
    : super(
        camera: CameraComponent.withFixedResolution(
          width: gameWidth,
          height: gameHeight,
        ),
      );

  final ValueNotifier<int> score = ValueNotifier(0); // Add this line
  final rand = math.Random();
  double get width => size.x;
  double get height => size.y;

  late PlayState _playState;
  PlayState get playState => _playState;
  set playState(PlayState playState) {
    _playState = playState;
    switch (playState) {
      case PlayState.welcome:
      case PlayState.gameOver:
      case PlayState.won:
        overlays.add(playState.name);
      case PlayState.playing:
        overlays.remove(PlayState.welcome.name);
        overlays.remove(PlayState.gameOver.name);
        overlays.remove(PlayState.won.name);
    }
  }

  @override
  FutureOr<void> onLoad() async {
    super.onLoad();

    camera.viewfinder.anchor = Anchor.topLeft;

    world.add(PlayArea());
  }
}

```



```

    playState = PlayState.welcome;
}

void startGame() {
    if (playState == PlayState.playing) return;

    world.removeAll(world.children.query<Ball>());
    world.removeAll(world.children.query<Bat>());
    world.removeAll(world.children.query<Brick>());

    playState = PlayState.playing;
    score.value = 0; // Add this line

    world.add(
        Ball(
            difficultyModifier: difficultyModifier,
            radius: ballRadius,
            position: size / 2,
            velocity: Vector2(
                (rand.nextDouble() - 0.5) * width,
                height * 0.2,
            ).normalized()..scale(height / 4),
        ),
    );

    world.add(
        Bat(
            size: Vector2(batWidth, batHeight),
            cornerRadius: const Radius.circular(ballRadius / 2),
            position: Vector2(width / 2, height * 0.95),
        ),
    );

    world.addAll([
        for (var i = 0; i < brickColors.length; i++)
        for (var j = 1; j <= 5; j++)
        Brick(
            position: Vector2(
                (i + 0.5) * brickWidth + (i + 1) * brickGutter,
                (j + 2.0) * brickHeight + j * brickGutter,
            ),
            color: brickColors[i],
        ),
    ]);
}

@override
void onTap() {
    super.onTap();
    startGame();
}

```

```

@override
KeyEventResult onKeyEvent(
  KeyEvent event,
  Set<LogicalKeyboardKey> keysPressed,
) {
  super.onKeyEvent(event, keysPressed);
  switch (event.logicalKey) {
    case LogicalKeyboardKey.arrowLeft:
      world.children.query<Bat>().first.moveBy(-batStep);
    case LogicalKeyboardKey.arrowRight:
      world.children.query<Bat>().first.moveBy(batStep);
    case LogicalKeyboardKey.space:
    case LogicalKeyboardKey.enter:
      startGame();
  }
  return KeyEventResult.handled;
}

@override
Color backgroundColor() => const Color(0xffff2e8cf);
}

```

在遊戲中加入 `score`，即可將遊戲狀態繫結至 Flutter 狀態管理。

2. 修改 `Brick` 類別，在玩家打破磚塊時加分。

[lib/src/components/brick.dart](#)

```

import 'package:flame/collisions.dart';
import 'package:flame/components.dart';
import 'package:flutter/material.dart';

import '../brick_breaker.dart';
import '../config.dart';
import 'ball.dart';
import 'bat.dart';

class Brick extends RectangleComponent
  with CollisionCallbacks, HasGameReference<BrickBreaker> {
  Brick({required super.position, required Color color})
    : super(
        size: Vector2(brickWidth, brickHeight),
        anchor: Anchor.center,
        paint: Paint()
          ..color = color
          ..style = PaintingStyle.fill,
        children: [RectangleHitbox()],
      );

  @override
  void onCollisionStart(
    Set<Vector2> intersectionPoints,
    PositionComponent other,
  ) {
    super.onCollisionStart(intersectionPoints, other);
    removeFromParent();
    game.score.value++; // Add this line

    if (game.world.children.query<Brick>().length == 1) {
      game.playState = PlayState.won;
      game.world.removeAll(game.world.children.query<Ball>());
      game.world.removeAll(game.world.children.query<Bat>());
    }
  }
}

```

製作美觀的遊戲

現在您可以在 Flutter 中記錄分數，接著就要將小工具組合起來，讓畫面看起來更美觀。

1. 在 `lib/src/widgets` 中建立 `score_card.dart`，並新增下列項目。

[lib/src/widgets/score_card.dart](#)

```
import 'package:flutter/material.dart';

class ScoreCard extends StatelessWidget {
  const ScoreCard({super.key, required this.score});

  final ValueNotifier<int> score;

  @override
  Widget build(BuildContext context) {
    return ValueListenableBuilder<int>(
      valueListenable: score,
      builder: (context, score, child) {
        return Padding(
          padding: const EdgeInsets.fromLTRB(12, 6, 12, 18),
          child: Text(
            'Score: $score'.toUpperCase(),
            style: Theme.of(context).textTheme.titleLarge!,
          ),
        );
      },
    );
  }
}
```

2. 在 `lib/src/widgets` 中建立 `overlay_screen.dart`，並加入下列程式碼。

這項功能會運用 `flutter_animate` 套件的強大功能，為疊加畫面加入動畫和樣式，讓疊加畫面更加精緻。

[lib/src/widgets/overlay_screen.dart](#)

```

import 'package:flutter/material.dart';
import 'package:flutter_animate/flutter_animate.dart';

class OverlayScreen extends StatelessWidget {
  const OverlayScreen({super.key, required this.title, required this.subtitle});

  final String title;
  final String subtitle;

  @override
  Widget build(BuildContext context) {
    return Container(
      alignment: const Alignment(0, -0.15),
      child: Column(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          Text(
            title,
            style: Theme.of(context).textTheme.headlineLarge,
          ).animate().slideY(duration: 750.ms, begin: -3, end: 0),
          const SizedBox(height: 16),
          Text(subtitle, style: Theme.of(context).textTheme.headlineSmall)
            .animate(onPlay: (controller) => controller.repeat())
            .fadeIn(duration: 1.seconds)
            .then()
            .fadeOut(duration: 1.seconds),
        ],
      ),
    );
  }
}

```

如要深入瞭解 `flutter_animate` 的強大功能，請參閱「[在 Flutter 中建構新一代 UI](#)」程式碼研究室。

這個程式碼在 `GameApp` 元件中變更了許多內容。首先，如要讓 `ScoreCard` 存取 `score`，請將 `score` 從 `StatelessWidget` 轉換為 `StatefulWidget`。如要新增評量表，必須新增 `Column`，將分數堆疊在遊戲上方。

其次，您新增了 `OverlayScreen` 小工具，以提升歡迎、遊戲結束和獲勝體驗。

[lib/src/widgets/game_app.dart](#)

```

import 'package:flame/game.dart';
import 'package:flutter/material.dart';
import 'package:google_fonts/google_fonts.dart';

import '../brick_breaker.dart';
import '../config.dart';
import 'overlay_screen.dart'; // Add this import
import 'score_card.dart';    // And this one too

class GameApp extends StatefulWidget { // Modify this line
  const GameApp({super.key});

  @override // Add from here...
  State<GameApp> createState() => _GameAppState();
}

class _GameAppState extends State<GameApp> {
  late final BrickBreaker game;

  @override
  void initState() {
    super.initState();
    game = BrickBreaker();
  } // To here.

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      theme: ThemeData(
        textTheme: GoogleFonts.pressStart2pTextTheme().apply(
          bodyColor: const Color(0xff184e77),
          displayColor: const Color(0xff184e77),
        ),
      ),
      home: Scaffold(
        body: Container(
          decoration: const BoxDecoration(
            gradient: LinearGradient(
              begin: Alignment.topCenter,
              end: Alignment.bottomCenter,
              colors: [Color(0xffa9d6e5), Color(0xffff2e8cf)],
            ),
          ),
          child: SafeArea(
            child: Padding(
              padding: const EdgeInsets.all(16),
              child: Center(
                child: Column( // Modify from here...
                  children: [
                    ScoreCard(score: game.score),
                    Expanded(

```

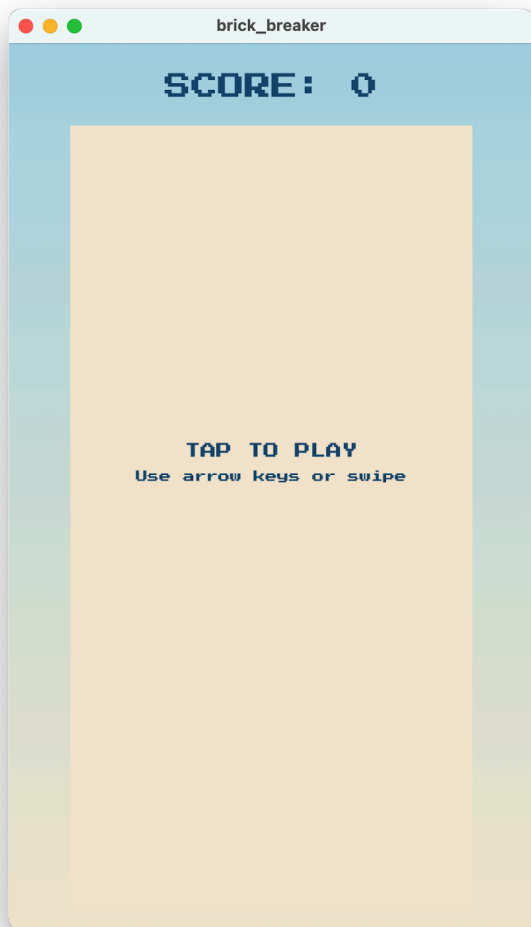
```

        child: FittedBox(
          child: SizedBox(
            width: gameWidth,
            height: gameHeight,
            child: GameWidget(
              game: game,
              overlayBuilderMap: {
                PlayState.welcome.name: (context, game) =>
                  const OverlayScreen(
                    title: 'TAP TO PLAY',
                    subtitle: 'Use arrow keys or swipe',
                  ),
                PlayState.gameOver.name: (context, game) =>
                  const OverlayScreen(
                    title: 'G A M E   O V E R',
                    subtitle: 'Tap to Play Again',
                  ),
                PlayState.won.name: (context, game) =>
                  const OverlayScreen(
                    title: 'Y O U   W O N ! ! !',
                    subtitle: 'Tap to Play Again',
                  ),
              },
            ),
          ),
        ),
      ),
    ],
  ),
),
),
),
),
),
),
),
),
),
),
),
);
}
}

```

// To here.

完成上述所有步驟後，您現在應該就能在六個 Flutter 目標平台中的任一平台上執行這款遊戲。遊戲畫面應如下所示。



11. 恭喜

恭喜，您已成功使用 Flutter 和 Flame 建構遊戲！

您使用 Flame 2D 遊戲引擎建構遊戲，並將其嵌入 Flutter 包裝函式。您使用 Flame 的效果來製作動畫和移除元件。您使用了 Google Fonts 和 Flutter Animate 套件，讓整個遊戲看起來設計精美。

後續步驟

查看一些程式碼研究室...

- [在 Flutter 中建構新一代 UI](#)
- [讓 Flutter 應用程式從無趣變為美觀](#)
- [在 Flutter 應用程式中加入應用程式內購](#)

延伸閱讀

- [Flutter 休閒遊戲工具包](#)
 - [Flame Engine](#)
 - [我們如何在短短六週內，以 Flutter 和 Flame 建構全新的 Super Dash 示範應用程式！](#)
-